VOLUMEN 06 · MÁSTER DATA & IA

Python para datos II

pandas, ficheros de datos, Parquet, APIs y gráficos. Las herramientas de datos de Python, sobre los mismos datos de Marejada, y cuándo usar cada una frente a DuckDB.

PARA QUIÉN

Álvaro, con el Volumen 05 terminado. Ya escribes Python; ahora vas a trabajar con tablas en Python.

CÓMO ESTÁ ESCRITO

Como el Volumen 05: cada celda se ha ejecutado de verdad en un notebook con Python 3.12 y pandas 3.0, las mismas versiones que tienes en tu proyecto, y su salida aparece tal cual. Cuando pandas y SQL hacen lo mismo de forma distinta, se ponen lado a lado.

INCLUYE

Seis tickets, entre ellos el cruce de la tienda web con el ERP que quedó pendiente en el Volumen 02, con la revisión de Elena, y una chuleta de pandas. Cierra el Bloque 0.

Preparado para Álvaro · Octubre de 2026 · Unos dos días de trabajo

Índice

PARTE 0 · ANTES DE EMPEZAR 3

1. Cómo trabajar con este volumen 4

PARTE 1 · PANDAS 6

2. Series y DataFrames 7

3. Leer los ficheros de Marejada 10

4. Seleccionar y filtrar 14

5. Transformar 17

6. Agregar 20

7. Unir 22

8. Series temporales 24

PARTE 2 · FICHEROS, APIS Y GRÁFICOS 26

9. Parquet 27

10. APIs 30

11. Gráficos con matplotlib 36

12. pandas y DuckDB, juntos 39

TICKETS · TICKETS DEL VOLUMEN 06 42

Tickets · Volumen 06 43

Soluciones · Volumen 06 45

APÉNDICES · PARA TENER A MANO 61

A. Chuleta de pandas 62

B. Qué viene en el Volumen 07 64

PARTE 0

Antes de empezar

Qué es pandas, qué lugar ocupa al lado de DuckDB y cómo preparar el entorno.

1. Cómo trabajar con este volumen

1. Cómo trabajar con este volumen

1.1 pandas, la tabla de Python

En el Volumen 05 trabajaste con listas de tuplas y diccionarios. Sirven, pero para una tabla de 380.000 filas son incómodos y lentos: cada suma es un bucle. **pandas** es la librería que da a Python una tabla de verdad, el **DataFrame**: filas y columnas con nombre y tipo, y operaciones (filtrar, agrupar, unir) que se aplican a toda la tabla de golpe.

Es la herramienta de datos más usada de Python desde hace quince años, y la vas a ver en todas partes: en los notebooks de Fabric y de Databricks, en los tutoriales de machine learning del Bloque 5, en las ofertas de trabajo. Y es, en el fondo, un SQL escrito con otra sintaxis. Lo vas a notar en cada capítulo.

1.2 pandas o DuckDB

Ya tienes un motor que hace todo esto muy bien: DuckDB. ¿Para qué otro? Porque hacen cosas distintas mejor:

	DUCKDB (SQL)	PANDAS (PYTHON)
Lo mejor que hace	Consultar, agregar y unir tablas grandes	Manipular una tabla dentro de un programa de Python: limpiarla fila a fila, pasarla a un gráfico, a un modelo, a un Excel
Volumen cómodo	Cientos de millones de filas	Hasta unos millones (todo vive en la memoria)
Velocidad	Muy alta	Alta en operaciones vectorizadas; baja si recorres filas en un bucle
Lo que esperan las demás librerías	Nada en particular	Casi todas (matplotlib, scikit-learn) reciben DataFrames

La respuesta corta: **agregar con DuckDB y terminar con pandas**. Y se pasan datos de uno a otro con una línea en cada dirección (capítulo 12). En el Bloque 3 conocerás un tercero, Spark, que tiene un DataFrame casi igual que el de pandas pero repartido entre varios ordenadores.

1.3 Preparar el entorno

pandas ya lo instalaste en el Volumen 01. Este volumen necesita dos paquetes más: `pyarrow`, que es lo que permite a pandas leer y escribir ficheros Parquet (capítulo 9), y `requests`, para las APIs (capítulo 10), que ya instalaste en el 8.5 del Volumen 05. Desde la raíz del proyecto:

TÚ ESCRIBES

```
uv add pyarrow
```

Y un notebook nuevo, `notebooks/06_pandas.ipynb`, con el kernel del `.venv`. La primera celda, siempre la misma:

PYTHON · EN TU NOTEBOOK

```
import pandas as pd
import duckdb

pd.__version__
```

RESULTADO

```
'3.0.6'
```

`import pandas as pd` es una convención universal: en cualquier código que leas, `pd` es pandas.

PARTE 1

pandas

La tabla de Python: crearla, leerla de los ficheros de Marejada, filtrarla, transformarla, agruparla y unirla. Cada operación, con su equivalente en SQL.

2. Series y DataFrames

3. Leer los ficheros de Marejada

4. Seleccionar y filtrar

5. Transformar

6. Agregar

7. Unir

8. Series temporales

2. Series y DataFrames

2.1 Una tabla desde DuckDB

La forma más rápida de tener un DataFrame con los datos de Marejada es pedírselo a DuckDB con `.df()`:

PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
productos = con.sql("SELECT * FROM gold.dim_producto WHERE producto_key > 0").df()
con.close()

productos.head()
```

RESULTADO

```
   producto_key  producto_id       sku               nombre categoria  \
0             1         1001  CER-0001     Faro Norte Lager  Cervezas
1             2         1002  CER-0002     Faro Norte Lager  Cervezas
2             3         1003  CER-0003     Faro Norte Lager  Cervezas
3             4         1004  CER-0004     Faro Norte Lager  Cervezas
4             5         1005  CER-0005  Faro Norte Especial  Cervezas

  subcategoria       marca             formato  precio_lista  coste_unitario  \
0        Lager  Faro Norte         Barril 30 L        102.00           70.33
1        Lager  Faro Norte         Barril 20 L         70.67           51.75
2        Lager  Faro Norte  Caja 24x33 cl lata         20.94           14.58
3        Lager  Faro Norte       Caja 12x75 cl         19.20           14.20
4     Especial  Faro Norte         Barril 30 L        117.77           79.49

   iva_pct  activo
0       21   False
1       21    True
2       21   False
3       21    True
4       21    True
```

`.head()` muestra las cinco primeras filas, como un `LIMIT 5`. En VS Code el DataFrame se ve como una tabla con formato; aquí aparece como texto, que es la otra forma en que pandas lo muestra, con las columnas que no caben sustituidas por `...`. La columna de la izquierda, sin nombre (`0, 1, 2...`), es el **índice**: la etiqueta de cada fila. Por ahora, piensa en él como el número de fila.

2.2 Lo primero con una tabla nueva

Las tres preguntas del Volumen 02 (¿cuántas filas?, ¿qué columnas y de qué tipo?, ¿qué valores?), en pandas:

PYTHON · EN TU NOTEBOOK

```
print(productos.shape)
productos.dtypes
```

RESULTADO

```
(235, 12)
producto_key        int32
producto_id         int64
sku                   str
nombre                str
categoria             str
subcategoria          str
marca                 str
formato               str
precio_lista      float64
coste_unitario    float64
iva_pct             int64
activo               bool
dtype: object
```

TIPO DE PANDAS	QUÉ ES	EN SQL
`int32`, `int64`	Enteros	`INTEGER`, `BIGINT`
`float64`	Decimales aproximados	`DOUBLE`
`str`	Texto (desde pandas 3.0; antes salía `object`)	`VARCHAR`
`bool`	Verdadero o falso	`BOOLEAN`
`datetime64[...]`	Fecha y hora	`TIMESTAMP`, `DATE`
`object`	"Cualquier cosa de Python". Si te aparece, suele ser una columna con tipos mezclados	Sin equivalente

Fíjate en `precio_lista` y `coste_unitario`. En gold son `DECIMAL(10,2)`, el tipo exacto para el dinero del Volumen 05 (2.3), y han llegado como `float64`. pandas no tiene un tipo decimal propio, y `.df()` los convierte a decimales aproximados. Para analizar da igual: los errores de redondeo están en el decimoquinto decimal. Para cuadrar una factura al céntimo, la suma se hace en DuckDB, con `DECIMAL`, y a pandas se lleva el resultado.

Y `.describe()`, el `SUMMARIZE` de pandas:

PYTHON · EN TU NOTEBOOK

```
productos[["iva_pct"]].describe()
```

RESULTADO

```
          iva_pct
count  235.000000
mean    18.659574
std      4.511508
min     10.000000
25%     21.000000
50%     21.000000
75%     21.000000
max     21.000000
```

2.3 Una columna es una Series

Cada columna de un DataFrame es una **Series**: una lista de valores con su índice y su tipo. Se saca con corchetes y el nombre:

PYTHON · EN TU NOTEBOOK

```
categorias = productos["categoria"]
print(type(categorias))
categorias.value_counts()
```

RESULTADO

```
<class 'pandas.Series'>
categoria
Cervezas                84
Refrescos               26
Vinos                   26
Licores y destilados    23
Alimentación            18
Limpieza e higiene      18
Café e infusiones       17
Aguas                   15
Desechables              8
Name: count, dtype: int64
```

`value_counts()` es el `GROUP BY categoria ... count() ORDER BY count() DESC` de una sola columna, y probablemente el método que más vas a usar para conocer unos datos.

CON CORCHETES	DEVUELVE	EJEMPLO
Un nombre: `df["categoria"]`	Una Series (una columna)	Para operar con ella
Una lista de nombres: `df[["sku", "nombre"]]`	Un DataFrame con esas columnas	Es el `SELECT sku, nombre`

2.4 Operaciones vectorizadas

Con una Series se opera **entera de golpe**, sin bucles. Es lo que hace a pandas rápido:

PYTHON · EN TU NOTEBOOK

```
precios = productos["precio_lista"]
print(precios.mean().round(2), precios.max())
(precios * 1.21).head(3)
```

RESULTADO

```
32.29 117.77
0    123.4200
1     85.5107
2     25.3374
Name: precio_lista, dtype: float64
```

`precios * 1.21` *multiplica los 235 precios a la vez, igual que un* `SELECT precio * 1.21` calcula toda la columna. Escribirlo con un `for` daría lo mismo, mucho más despacio. La regla de oro de pandas es la misma que la de SQL: **piensa en columnas, no en filas**.

3. Leer los ficheros de Marejada

3.1 CSV: read_csv

`pd.read_csv` tiene las mismas opciones que el `read_csv` de DuckDB del Volumen 01, con otros nombres. Los pedidos:

PYTHON · EN TU NOTEBOOK

```python
pedidos = pd.read_csv(
    "../datos/erp/pedidos.csv",
    sep=";",
    decimal=",",
    parse_dates=["fecha_pedido", "fecha_entrega"],
    date_format={"fecha_pedido": "%d/%m/%Y %H:%M", "fecha_entrega": "%d/%m/%Y"},
)
print(pedidos.shape)
pedidos.dtypes
```

RESULTADO

```
(77448, 10)
pedido_id                int64
fecha_pedido     datetime64[us]
cliente_id               int64
comercial_id             int64
canal                      str
estado                     str
fecha_entrega    datetime64[us]
almacen                    str
gastos_envio           float64
observaciones              str
dtype: object
```

DESMONTANDO EL COMANDO

`pd.read_csv("...", sep=";", decimal=",", parse_dates=[...], date_format={...})`

`sep=";"`	El separador. El `delim` de DuckDB.
`decimal=","`	La coma decimal. El `decimal_separator` de DuckDB.
`parse_dates=[...]`	Qué columnas convertir en fechas. Sin esto, se quedan como texto.
`date_format={...}`	El formato de cada una, con las letras `%` de siempre. Como en DuckDB, **no lo dejes adivinar**: `04/01/2024` sería el 1 de abril para un programa en inglés.

Fíjate en `fecha_entrega`: salió como fecha, aunque en el CSV hay pedidos sin entrega. Los huecos se convierten en `NaT` (*Not a Time*), el nulo de las fechas. En los números, el nulo es `NaN` (*Not a Number*); en los textos, `NaN` también. Los tres son el `NULL` de SQL, y se buscan igual: con `.isna()` (4.5).

3.2 Latin-1

PYTHON · EN TU NOTEBOOK

```
comerciales = pd.read_csv("../datos/rrhh/comerciales.csv", encoding="latin-1")
comerciales[["comercial_id", "nombre", "apellidos", "jefe_id"]].head(4)
```

RESULTADO

```
   comercial_id  nombre        apellidos  jefe_id
0             1   Íñigo       Sainz Ruiz      NaN
1             2  Raquel   Fernández Cobo      1.0
2             3  Daniel  Gutiérrez Pérez      2.0
3             4    Sara   Ceballos Ortiz      2.0
```

La misma `encoding="latin-1"` del Volumen 05. Y mira `jefe_id`: salió como `float64`, con `1.0` y `NaN`. pandas, por defecto, no tiene enteros con nulos: si una columna de números tiene algún hueco, la convierte a decimales para poder guardar el `NaN`. Para pedirle enteros con nulos hay un tipo especial, `Int64` (con mayúscula):

PYTHON · EN TU NOTEBOOK

```
comerciales["jefe_id"] = comerciales["jefe_id"].astype("Int64")
comerciales["jefe_id"].head(4)
```

RESULTADO

```
0    <NA>
1       1
2       2
3       2
Name: jefe_id, dtype: Int64
```

`<NA>` es el nulo de los tipos nuevos de pandas. Te lo encontrarás a menudo con identificadores que pueden faltar, como este o como el `cliente_maestro_id` de los clientes borrados.

3.3 Excel con títulos: read_excel

La hoja de Lucía, con su título de adorno en las filas 1 y 2 (Volumen 02, MJD-204). En DuckDB se le daba el rango; en pandas, cuántas filas saltarse:

PYTHON · EN TU NOTEBOOK

```
conta = pd.read_excel("../datos/contabilidad/facturacion_mensual_2024-2026.xlsx", skiprows=3)
print(conta.shape)
conta.head()
```

RESULTADO

```
(151, 5)
    Año      Mes  Provincia  Base imponible (€)  Gastos de envío incluidos (€)
0  2024    enero   Asturias           206720.18                            774
1  2024    enero     Burgos            76489.34                            612
2  2024    enero  Cantabria           474353.90                           2064
3  2024    enero   Palencia            53726.63                            180
4  2024  febrero   Asturias           152919.36                           1008
```

`skiprows=3` salta las tres primeras filas (título, subtítulo y la vacía), y la cuarta se toma como cabecera. pandas usa el paquete `openpyxl` del Volumen 01 para leer el Excel. Y, a diferencia de DuckDB con el rango `A4:E1000`, no se ha traído ninguna fila vacía del final: 151 filas, una por mes y provincia con ventas (Asturias y Bizkaia empiezan más tarde).

3.4 JSON anidado: json_normalize

El JSON de la tienda web es un diccionario con una lista de pedidos, cada uno con un cliente dentro y una lista de líneas dentro. pandas lo aplana con `json_normalize`:

PYTHON · EN TU NOTEBOOK

```
import json

with open("../datos/web/pedidos_web.json", encoding="utf-8") as f:
    web = json.load(f)

pedidos_web = pd.json_normalize(web["orders"])
print(pedidos_web.shape)
pedidos_web.drop(columns="items").head(3)
```

RESULTADO

```
(7956, 8)
     order_id            created_at  shipping     status  erp_pedido_id  \
0  WEB-000001  2025-04-07T04:04:00Z       0.0  completed   2.025006e+09
1  WEB-000002  2025-04-07T15:00:00Z       0.0  completed   2.025006e+09
2  WEB-000003  2025-04-07T16:39:00Z       0.0  completed   2.025006e+09

  customer.nif           customer.name
0    B43996889       Bar La Castellana
1    B45715312  Restaurante La Pradera
2    B41082697    Catering La Barquera
```

El cliente, que era un diccionario dentro de cada pedido, se ha convertido en dos columnas con el nombre compuesto: `customer.nif` y `customer.name`. Las líneas (`items`) se quedan como una lista dentro de cada fila, porque cada pedido tiene varias: para aplanarlas hay que decir que la tabla es **de líneas**:

PYTHON · EN TU NOTEBOOK

```
lineas_web = pd.json_normalize(
    web["orders"],
    record_path="items",
    meta=["order_id", "erp_pedido_id", "status"],
)
print(lineas_web.shape)
lineas_web.head(3)
```

RESULTADO

```
(38825, 7)
        sku  qty  unit_price  discount    order_id erp_pedido_id     status
0  AGU-0014    1        9.13       5.0  WEB-000001    2025005606  completed
1  CER-0071    2       70.16       5.0  WEB-000001    2025005606  completed
2  LIM-0007   10       15.24       5.0  WEB-000001    2025005606  completed
```

DESMONTANDO EL COMANDO

`pd.json_normalize(web["orders"], record_path="items", meta=[...])`

`web["orders"]`	La lista de pedidos.
`record_path="items"`	Cada fila del resultado será un elemento de `items`: una línea. Es el `unnest` de DuckDB.
`meta=[...]`	Campos del pedido que se copian en cada una de sus líneas, para saber a qué pedido pertenece. Es repetir la cabecera en cada línea: el grano del Volumen 04 en una sola tabla.

3.5 Lo que se puede leer

FORMATO	FUNCIÓN	OBSERVACIONES
CSV	`pd.read_csv`	`sep`, `decimal`, `encoding`, `parse_dates`, `date_format`, `dtype`
Excel	`pd.read_excel`	`sheet_name`, `skiprows`, `usecols`. Usa openpyxl
JSON	`pd.read_json` o `json.load` + `pd.json_normalize`	Para JSON anidado, la segunda
Parquet	`pd.read_parquet`	Capítulo 9. Usa pyarrow
SQL / DuckDB	`con.sql("...").df()`	Lo más rápido para datos que ya están en una base

4. Seleccionar y filtrar

4.1 Filtrar filas: máscaras

En pandas, el `WHERE` se escribe en dos pasos. Primero una condición sobre una columna, que devuelve una Series de verdaderos y falsos (una **máscara**); después, se usa la máscara para quedarse con las filas verdaderas:

PYTHON · EN TU NOTEBOOK

```
cancelados = pedidos["estado"] == "Cancelado"
print(cancelados.head(3))
print(cancelados.sum())
pedidos[cancelados].shape
```

RESULTADO

```
0    False
1    False
2    False
Name: estado, dtype: bool
2732
(2732, 10)
```

`cancelados.sum()` cuenta los `True` (cada `True` vale 1): 2.732, la cifra de control. Y `pedidos[cancelados]` devuelve solo esas filas. Lo normal es escribirlo en una línea: `pedidos[pedidos["estado"] == "Cancelado"]`.

4.2 Varias condiciones y la trampa de los paréntesis

Las condiciones se combinan con `&` (y), `|` (o) y `~` (no), **no** con `and`, `or` y `not`. Y cada condición va entre paréntesis. Sin ellos:

PYTHON · EN TU NOTEBOOK

```
pedidos[pedidos["canal"] == "Web B2B" & pedidos["estado"] == "Cancelado"]
```

ERROR

```
---------------------------------------------------------------------------
ArrowNotImplementedError                  Traceback (most recent call last)
Cell In[14], line 1
----> 1 pedidos[pedidos["canal"] == "Web B2B" & pedidos["estado"] == "Cancelado"]

(...)

TypeError: operation 'rand_' not supported for dtype 'str' with object of type <class 'str'>
```

El mensaje (aquí recortado: el original ocupa una pantalla entera) es confuso, y la causa es la prioridad de operadores, como el `AND` y el `OR` del Volumen 02 (4.2): en Python, `&` se calcula antes que `==`, así que intenta hacer `"Web B2B" & pedidos["estado"]` y no sabe. Con paréntesis:

PYTHON · EN TU NOTEBOOK

```
web_cancelados = pedidos[(pedidos["canal"] == "Web B2B") & (pedidos["estado"] == "Cancelado")]
len(web_cancelados)
```

RESULTADO

```
297
```

IDEA CLAVE

En pandas, cada condición entre paréntesis y unidas con `&`, `|` o `~`. Si ves un error que habla de *truth value of a Series is ambiguous* o de tipos que no se pueden combinar con `&`, casi siempre son los paréntesis o un `and` donde iba `&`.

4.3 isin, between y query

PYTHON · EN TU NOTEBOOK

```
print(pedidos["canal"].isin(["Teléfono", "Comercial"]).sum())
print(pedidos["fecha_pedido"].between("2025-03-01", "2025-03-31 23:59").sum())
pedidos.query("canal == 'Web B2B' and estado == 'Cancelado'").shape
```

RESULTADO

```
69523
1908
(297, 10)
```

`isin` es el `IN` de SQL; `between`, el `BETWEEN` (con las mismas trampas de las fechas con hora del Volumen 02: por eso el `23:59`, y mejor `>=` y `<`). Y `query` permite escribir la condición como un texto, con `and` y `or` normales: más legible para filtros largos.

4.4 Seleccionar filas y columnas a la vez: loc

`.loc[filas, columnas]` combina las dos cosas, y es la forma recomendada cuando además vas a modificar:

PYTHON · EN TU NOTEBOOK

```
pedidos.loc[pedidos["canal"] == "Web B2B", ["pedido_id", "fecha_pedido", "estado"]].head(3)
```

RESULTADO

```
       pedido_id        fecha_pedido           estado
29653  2025005606 2025-04-07 06:04:00  Devuelto parcial
29748  2025005701 2025-04-07 17:00:00        Entregado
29758  2025005711 2025-04-07 18:39:00        Entregado
```

Hay un hermano, `.iloc`, que selecciona por **posición** (`df.iloc[0]` es la primera fila, `df.iloc[:, 2]` la tercera columna). Se usa poco en análisis, porque las posiciones cambian al ordenar o filtrar y los nombres no.

4.5 Nulos

PYTHON · EN TU NOTEBOOK

```
print(pedidos["fecha_entrega"].isna().sum())
print(pedidos["observaciones"].notna().sum())
pedidos.isna().sum()
```

RESULTADO

```
3004
4662
pedido_id            0
fecha_pedido         0
cliente_id           0
comercial_id         0
canal                0
estado               0
fecha_entrega     3004
almacen              0
gastos_envio         0
observaciones    72786
dtype: int64
```

`isna()` es el `IS NULL`; `notna()`, el `IS NOT NULL`. Y `pedidos.isna().sum()` cuenta los nulos de **todas** las columnas a la vez: la columna `null_percentage` del `SUMMARIZE` en una línea. 3.004 pedidos sin fecha de entrega: los cancelados, los pendientes y los que están en reparto.

5. Transformar

5.1 Columnas nuevas

Asignar a un nombre de columna que no existe la crea, como un `SELECT ... AS` que se queda en la tabla:

PYTHON · EN TU NOTEBOOK

```
lineas = pd.read_csv("../datos/erp/lineas_pedido.csv", sep=";", decimal=",")
lineas["importe_neto"] = lineas["cantidad"] * lineas["precio_unitario"] * (1 -
lineas["descuento_pct"] / 100)
lineas.head(3)
```

RESULTADO

```
   linea_id   pedido_id  num_linea  producto_id  cantidad  precio_unitario  \
0         1  2024000001          1         1192         6            41.65
1         2  2024000001          2         1073         2            95.85
2         3  2024000002          1         1020         3            95.49

   descuento_pct  importe_neto
0              0      249.9000
1              0      191.7000
2             15      243.4995
```

5.2 Texto: el accesor .str

Los métodos de texto del Volumen 05, aplicados a toda una columna, van detrás de `.str`:

PYTHON · EN TU NOTEBOOK

```
clientes = pd.read_csv("../datos/erp/clientes.csv", sep=";")
provincia_norm = clientes["provincia"].str.strip().str.lower()
provincia_norm.value_counts()
```

RESULTADO

```
provincia
cantabria                703
asturias                 181
bizkaia                  123
burgos                    96
palencia                  62
vizcaya                   25
principado de asturias    16
santander                  6
Name: count, dtype: int64
```

Las quince grafías del Volumen 02 reducidas a ocho. Para terminar la limpieza, la tabla de correspondencias de silver se escribe como un diccionario, y `.map` lo aplica:

PYTHON · EN TU NOTEBOOK

```python
correspondencias = {
    "cantabria": "Cantabria", "santander": "Cantabria",
    "asturias": "Asturias", "principado de asturias": "Asturias",
    "bizkaia": "Bizkaia", "vizcaya": "Bizkaia",
    "burgos": "Burgos", "palencia": "Palencia",
}
clientes["provincia_limpia"] = provincia_norm.map(correspondencias).fillna("REVISAR")
clientes["provincia_limpia"].value_counts()
```

RESULTADO

```
provincia_limpia
Cantabria    709
Asturias     197
Bizkaia      148
Burgos        96
Palencia      62
Name: count, dtype: int64
```

DESMONTANDO EL COMANDO

`provincia_norm.map(correspondencias).fillna("REVISAR")`

`.map(diccionario)`	Cambia cada valor por el que le corresponde en el diccionario. Lo que no esté en el diccionario se convierte en nulo. Es el `LEFT JOIN` con `silver.map_provincias`.
`.fillna("REVISAR")`	Rellena los nulos. Es el `coalesce(..., 'REVISAR')` de silver. Ninguno ha hecho falta: todas encajan.

5.3 Fechas: el accesor .dt

Igual que `.str`, las fechas tienen `.dt`:

PYTHON · EN TU NOTEBOOK

```python
pedidos["anio"] = pedidos["fecha_pedido"].dt.year
pedidos["mes"] = pedidos["fecha_pedido"].dt.to_period("M")
pedidos["dia_semana"] = pedidos["fecha_pedido"].dt.dayofweek
pedidos["dias_entrega"] = (pedidos["fecha_entrega"] - pedidos["fecha_pedido"].dt.normalize()).dt.days
pedidos[["fecha_pedido", "anio", "mes", "dia_semana", "dias_entrega"]].head(3)
```

RESULTADO

```
         fecha_pedido  anio      mes  dia_semana  dias_entrega
0 2024-01-01 08:03:00  2024  2024-01           0           3.0
1 2024-01-01 08:21:00  2024  2024-01           0           1.0
2 2024-01-01 08:23:00  2024  2024-01           0           2.0
```

PANDAS	SQL (DUCKDB)
`.dt.year`, `.dt.month`, `.dt.quarter`	`year()`, `month()`, `quarter()`
`.dt.to_period("M")`	`date_trunc('month', ...)`, como "periodo" (`2024-01`)
`.dt.dayofweek`	`isodow() - 1`: **en pandas el lunes es 0**, no 1
`.dt.normalize()`	`CAST(... AS DATE)`: quita la hora
Restar fechas y `.dt.days`	`fecha2 - fecha1`

5.4 Cambiar el tipo

`astype` es el `CAST`. El `pedido_id` de Marejada lleva el año en sus cuatro primeras cifras (`2024000001`); pasarlo a texto permite cortarlo:

PYTHON · EN TU NOTEBOOK

```
anio_del_id = pedidos["pedido_id"].astype(str).str[:4].astype(int)
print(anio_del_id.dtype)
(anio_del_id == pedidos["anio"]).all()
```

RESULTADO

```
int64
np.True_
```

Y para convertir texto en número cuando puede haber valores que no lo son, `pd.to_numeric(serie, errors="coerce")`: lo que no se pueda convertir queda como `NaN`. Es el `TRY_CAST`.

5.5 La trampa de la copia

Esta trampa te la vas a encontrar en cuanto filtres y modifiques:

PYTHON · EN TU NOTEBOOK

```
web = pedidos[pedidos["canal"] == "Web B2B"]
web["portes_con_iva"] = web["gastos_envio"] * 1.21
web.shape
```

RESULTADO

```
(7925, 15)
```

En pandas 3.0 funciona sin avisos, pero conviene saber **qué** ha pasado, porque cambió en esta versión y te encontrarás muchísimo código antiguo. `web` es una tabla nueva hecha a partir de `pedidos`; la columna se ha añadido a `web` y **`pedidos` no ha cambiado**:

PYTHON · EN TU NOTEBOOK

```
"portes_con_iva" in pedidos.columns
```

RESULTADO

```
False
```

En versiones anteriores de pandas, esto mismo daba un aviso famoso (`SettingWithCopyWarning`) porque no estaba claro si se modificaba solo el filtrado o también el original. pandas 3.0 lo resolvió con una regla simple, llamada *Copy-on-Write*: **un filtro siempre se comporta como una copia**. Si en un tutorial ves `.copy()` después de cada filtro, es por eso: era la forma de evitar el aviso. Con tu versión, no hace falta.

6. Agregar

6.1 groupby

El `GROUP BY`. Se agrupa por una o varias columnas y se dice qué calcular:

PYTHON · EN TU NOTEBOOK

```python
pedidos.groupby("canal")["gastos_envio"].sum()
```

RESULTADO

```
canal
Comercial    127926.0
Teléfono      54576.0
Web B2B       20232.0
Name: gastos_envio, dtype: float64
```

Varias medidas a la vez, cada una con su nombre, con `agg`:

PYTHON · EN TU NOTEBOOK

```python
resumen = (
    pedidos[pedidos["estado"] != "Cancelado"]
    .groupby(["anio", "canal"])
    .agg(
        pedidos=("pedido_id", "count"),
        clientes=("cliente_id", "nunique"),
        portes=("gastos_envio", "sum"),
    )
)
resumen
```

RESULTADO

```
              pedidos  clientes   portes
anio canal
2024 Comercial    16347       866  44130.0
     Teléfono      6864       848  18240.0
2025 Comercial    17291      1021  44616.0
     Teléfono      7652       997  19566.0
     Web B2B       2277       839   5952.0
2026 Comercial    13290      1099  34440.0
     Teléfono      5644      1071  14850.0
     Web B2B       5351      1068  13518.0
```

DESMONTANDO EL COMANDO

`df.groupby([...]).agg(nombre=("columna", "función"))`

`pedidos[pedidos["estado"] != "Cancelado"]`	Primero el filtro: el `WHERE`.
`.groupby(["anio", "canal"])`	El `GROUP BY anio, canal`.
`pedidos=("pedido_id", "count")`	`count(pedido_id) AS pedidos`. A la izquierda, el nombre de la columna resultante; a la derecha, la columna de origen y la función.
`"nunique"`	`count(DISTINCT ...)`.
`( ... )`	Todo entre paréntesis para poder partir la cadena en varias líneas, una operación por línea, como las cláusulas de una consulta SQL.

El resultado tiene un **índice de dos niveles** (`anio` y `canal` a la izquierda, en negrita en VS Code): los grupos se convierten en el índice. Para tenerlos como columnas normales, como en SQL, `.reset_index()`.

6.2 La tabla dinámica: pivot_table

Los trimestres en columnas del MJD-207, que en SQL necesitaron un `FILTER` por columna:

PYTHON · EN TU NOTEBOOK

```
validas = pedidos[pedidos["estado"] != "Cancelado"]
pd.pivot_table(
    validas,
    index="anio",
    columns="canal",
    values="pedido_id",
    aggfunc="count",
)
```

RESULTADO

```
canal  Comercial  Teléfono  Web B2B
anio
2024     16347.0    6864.0      NaN
2025     17291.0    7652.0   2277.0
2026     13290.0    5644.0   5351.0
```

Es la tabla dinámica de Excel: `index` va a las filas, `columns` a las columnas y `values` con `aggfunc` dentro de las celdas. Fíjate en el `NaN` de la web en 2024: no hay pedidos web, y pandas lo deja vacío, como el `FILTER` de SQL. Con `fill_value=0` saldría un cero.

6.3 Ordenar y los primeros

PYTHON · EN TU NOTEBOOK

```
(
    validas.groupby("cliente_id")["pedido_id"].count()
    .sort_values(ascending=False)
    .head(5)
)
```

RESULTADO

```
cliente_id
11108    346
10346    241
11057    221
10769    209
10298    195
Name: pedido_id, dtype: int64
```

`sort_values` es el `ORDER BY`, y `ascending=False` el `DESC`. Y para el top N de cada grupo (el 5.2 del Volumen 03), hay `groupby(...).head(n)` después de ordenar, o `rank`, igual que en SQL.

7. Unir

7.1 merge

El `JOIN` de pandas se llama `merge`. Las líneas con su pedido, para tener la fecha y el estado:

PYTHON · EN TU NOTEBOOK

```
lineas_con_pedido = lineas.merge(
    pedidos[["pedido_id", "fecha_pedido", "estado", "canal"]],
    on="pedido_id",
    how="left",
    validate="many_to_one",
)
print(len(lineas), len(lineas_con_pedido))
lineas_con_pedido.head(3)
```

RESULTADO

```
381158 381158
   linea_id   pedido_id  num_linea  producto_id  cantidad  precio_unitario  \
0         1  2024000001          1         1192         6            41.65
1         2  2024000001          2         1073         2            95.85
2         3  2024000002          1         1020         3            95.49

   descuento_pct  importe_neto        fecha_pedido     estado      canal
0              0      249.9000 2024-01-01 08:03:00  Entregado  Comercial
1              0      191.7000 2024-01-01 08:03:00  Entregado  Comercial
2             15      243.4995 2024-01-01 08:21:00  Entregado  Comercial
```

DESMONTANDO EL COMANDO

`lineas.merge(pedidos[...], on="pedido_id", how="left", validate="many_to_one")`

`lineas.merge(...)`	La tabla de la izquierda es la que llama a `merge`.
`on="pedido_id"`	La columna de unión. Si se llama distinto en cada tabla: `left_on=` y `right_on=`.
`how="left"`	`LEFT JOIN`. Las otras: `"inner"` (la de por defecto), `"right"`, `"outer"` (el `FULL JOIN`).
`validate="many_to_one"`	Comprueba que en la tabla de la derecha cada `pedido_id` aparece **una sola vez**. Si no, da error en lugar de multiplicar filas. Es la protección contra el abanico del Volumen 02 (8.6), y SQL no la tiene.

Las mismas 381.158 filas antes y después: el `LEFT JOIN` no ha perdido ni multiplicado nada, y `validate` lo garantiza. Fíjate en lo que pasaría al revés, con una tabla que **sí** tiene repetidos:

PYTHON · EN TU NOTEBOOK

```
lineas.merge(lineas[["pedido_id", "num_linea"]], on="pedido_id", validate="many_to_one")
```

ERROR

```
---------------------------------------------------------------------------
MergeError                                Traceback (most recent call last)
Cell In[31], line 1
----> 1 lineas.merge(lineas[["pedido_id", "num_linea"]], on="pedido_id", validate="many_to_one")

(...)

MergeError: Merge keys are not unique in right dataset; not a many-to-one merge

Duplicates in right:
  pedido_id
2024000001
2024000002
2024000002
```

El error dice exactamente qué ha fallado: las claves de la derecha no son únicas. Mejor un error ahora que unas ventas multiplicadas en un informe.

7.2 El anti-join: indicator

Las 36 líneas huérfanas del Volumen 02 (9.7), con el parámetro `indicator`, que añade una columna diciendo de qué tabla vino cada fila:

PYTHON · EN TU NOTEBOOK

```
cruce = lineas.merge(pedidos[["pedido_id"]], on="pedido_id", how="left", indicator=True)
cruce["_merge"].value_counts()
```

RESULTADO

```
_merge
both          381122
left_only         36
right_only         0
Name: count, dtype: int64
```

`left_only` son las filas de la izquierda sin pareja: el anti-join. `both`, las que encontraron pedido. `right_only` aparecería con un `how="outer"` para las de la derecha sin pareja. Es la herramienta perfecta para comparar dos fuentes, y la usarás en el MJD-601.

7.3 concat: apilar

`pd.concat` pone tablas una debajo de otra, como `UNION ALL`:

PYTHON · EN TU NOTEBOOK

```
ene = validas[validas["mes"] == "2026-01"]
feb = validas[validas["mes"] == "2026-02"]
print(len(ene), len(feb), len(pd.concat([ene, feb])))
```

RESULTADO

```
2004 1836 3840
```

8. Series temporales

8.1 resample: agregar por periodo

Para agregar por periodos de tiempo, pandas tiene una herramienta específica: con la fecha como índice, `resample` agrupa por mes, semana o día:

PYTHON · EN TU NOTEBOOK

```
ventas_dia = (
    lineas_con_pedido[lineas_con_pedido["estado"] != "Cancelado"]
    .set_index("fecha_pedido")["importe_neto"]
)
mensual = ventas_dia.resample("MS").sum().round()
mensual.tail(4)
```

RESULTADO

```
fecha_pedido
2026-06-01    1808130.0
2026-07-01    2146432.0
2026-08-01    1980108.0
2026-09-01    1574654.0
Name: importe_neto, dtype: float64
```

`"MS"` significa *month start*: un grupo por mes, etiquetado con el primer día. Pero fíjate en las cifras: no son las de gold. Septiembre de 2026 sale con 1.574.654 €, y en gold (lo verás en el apartado siguiente) son 1.571.643. La diferencia es que este cálculo parte de los CSV de bronze, **con las líneas duplicadas** y el resto de la suciedad. Es la trampa del Volumen 02 otra vez, y la razón de que la limpieza se haga una vez, en silver, y no en cada notebook. Para analizar, se parte de silver o de gold.

8.2 Ventanas: rolling y shift

Las funciones de ventana del Volumen 03, en pandas:

PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
mensual = con.sql("""
    SELECT d.inicio_mes AS mes, sum(f.importe_neto) AS ventas
    FROM gold.fact_ventas AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
    GROUP BY d.inicio_mes
    ORDER BY d.inicio_mes
""").df().set_index("mes")["ventas"]
con.close()

tabla = pd.DataFrame({
    "ventas": mensual,
    "media_3m": mensual.rolling(3).mean(),
    "mes_anterior": mensual.shift(1),
    "interanual_pct": (mensual / mensual.shift(12) - 1) * 100,
}).round(1)
tabla.tail(4)
```

RESULTADO

mes	ventas	media_3m	mes_anterior	interanual_pct
2026-06-01	1805025.7	1629220.5	1582346.1	33.4
2026-07-01	2142228.1	1843200.0	1805025.7	24.6
2026-08-01	1975639.4	1974297.7	2142228.1	18.1
2026-09-01	1571643.1	1896503.5	1975639.4	6.6

PANDAS	SQL (VOLUMEN 03)
`.rolling(3).mean()`	`avg(x) OVER (ORDER BY mes ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)`
`.shift(1)`	`lag(x) OVER (ORDER BY mes)`
`.shift(12)`	`lag(x, 12)` : con la misma trampa de los huecos (8.4 del Volumen 03)
`.cumsum()`	`sum(x) OVER (ORDER BY mes)`
`.pct_change()`	`x / lag(x) - 1`

Una diferencia con SQL: `rolling(3)` deja vacíos los dos primeros meses en lugar de calcular la media con menos meses, que era la trampa del 6.2 del Volumen 03. Aquí, por defecto, se hace lo prudente.

PARTE 2

Ficheros, APIs y gráficos

Parquet, el formato de las plataformas de datos; pedir datos a un servicio web; dibujar lo que has calculado; y cuándo usar pandas y cuándo DuckDB.

9. Parquet

10. APIs

11. Gráficos con matplotlib

12. pandas y DuckDB, juntos

9. Parquet

9.1 Qué es y por qué existe

Un CSV es texto: cada número se escribe con sus cifras, cada fila en una línea, y quien lo lee tiene que adivinar (o que le digas) el separador, los decimales, las fechas y la codificación. Has pasado medio Volumen 01 peleando con eso.

Parquet es un formato de fichero binario pensado para datos analíticos, y resuelve esos problemas de golpe:

	CSV	PARQUET
Tipos	No los guarda: todo es texto	Los guarda: un entero es un entero, una fecha es una fecha
Organización	Por filas	**Por columnas**: los valores de cada columna van juntos
Tamaño	Grande	Pequeño: comprime muy bien, porque los valores de una columna se parecen
Leer solo unas columnas	Hay que leer el fichero entero	Lee solo las columnas que pides
Lo abre Excel	Sí	No

Es el formato que hay debajo de casi todo lo que viene en el máster: las tablas Delta de Fabric y de Databricks son ficheros Parquet con un registro de cambios al lado, y en AWS es el formato recomendado para consultar datos en S3 con Athena. Conocerlo ahora te ahorra sorpresas después.

9.2 Escribir y leer

La tabla de hechos de ventas, de gold a Parquet. Los ficheros exportados van a `datos/exportaciones/`, que está dentro de `datos/` y por tanto fuera de Git (el `.gitignore` del Volumen 01):

PYTHON · EN TU NOTEBOOK

```python
from pathlib import Path

con = duckdb.connect("../db/marejada.duckdb", read_only=True)
ventas = con.sql("SELECT * FROM gold.fact_ventas").df()
con.close()

carpeta = Path("../datos/exportaciones")
carpeta.mkdir(exist_ok=True)

ventas.to_parquet(carpeta / "fact_ventas.parquet", index=False)
ventas.to_csv(carpeta / "fact_ventas.csv", index=False)

for f in sorted(carpeta.glob("fact_ventas.*")):
    print(f.name, round(f.stat().st_size / 1024 / 1024, 1), "MB")
```

RESULTADO

```
fact_ventas.csv 29.8 MB
fact_ventas.parquet 4.5 MB
```

DESMONTANDO EL COMANDO

`ventas.to_parquet(ruta, index=False)`

`to_parquet`	Escribe el DataFrame en Parquet. pandas usa para ello el paquete `pyarrow`, que instalaste en el 1.3. Sin él, el error dice exactamente eso: que falta `pyarrow`.
`index=False`	No guardar el índice de pandas (0, 1, 2…) como una columna más. Casi siempre lo quieres así; sin ello, a veces aparece una columna `__index_level_0__` al leerlo en otro sitio.
`carpeta.mkdir(exist_ok=True)`	Crea la carpeta, y si ya existe no da error (`pathlib`, Volumen 05, 7.1).
`f.stat().st_size`	El tamaño del fichero en bytes.

El mismo contenido: 29,8 MB en CSV y 4,5 MB en Parquet, menos de la sexta parte. Y al leerlo de vuelta, los tipos vienen solos, sin decirle nada:

PYTHON · EN TU NOTEBOOK

```
leido = pd.read_parquet(carpeta / "fact_ventas.parquet")
print(leido.shape)
print(leido.dtypes.head(6))

desde_csv = pd.read_csv(carpeta / "fact_ventas.csv")
print(desde_csv.dtypes.head(6))
```

RESULTADO

```
(366928, 14)
pedido_id        int64
num_linea        int64
fecha_key        int32
cliente_key      int32
producto_key     int32
comercial_key    int32
dtype: object
pedido_id        int64
num_linea        int64
fecha_key        int64
cliente_key      int64
producto_key     int64
comercial_key    int64
dtype: object
```

Fíjate en las claves. En gold son `INTEGER`, y llegaron a pandas como `int32`: el Parquet lo ha guardado y lo devuelve igual. El CSV solo tiene texto, y pandas, al leerlo, elige `int64` por su cuenta. Aquí la diferencia es pequeña, pero piensa en una columna de fechas o en un código postal de Burgos con su cero delante: el CSV obliga a volver a explicarlo todo; el Parquet se acuerda.

9.3 Leer solo lo que necesitas

Como Parquet guarda por columnas, pedir solo dos columnas lee solo esas dos:

PYTHON · EN TU NOTEBOOK

```
dos = pd.read_parquet(carpeta / "fact_ventas.parquet", columns=["fecha_key", "importe_neto"])
dos.shape
```

RESULTADO

```
(366928, 2)
```

Con 14 columnas no se nota. Con las tablas de cientos de columnas y miles de millones de filas de una empresa grande, es la diferencia entre leer 2 GB o 200.

9.4 DuckDB lee Parquet directamente

Y lo mejor: DuckDB consulta un fichero Parquet como si fuera una tabla, sin cargarlo antes:

PYTHON · EN TU NOTEBOOK

```
duckdb.sql("""
    SELECT count(*) AS lineas, round(sum(importe_neto), 2) AS ventas
    FROM '../datos/exportaciones/fact_ventas.parquet'
""")
```

RESULTADO

```
┌─────────┬──────────────┐
│ lineas  │    ventas    │
│ int64   │    double    │
├─────────┼──────────────┤
│ 366928  │ 42633273.34  │
└─────────┴──────────────┘
```

Esto lo vas a ver en el Bloque 2 y en el 3, con otra sintaxis: en Fabric y en Databricks, una consulta SQL sobre una carpeta de ficheros Parquet es el pan de cada día.

> **NOTA**
>
> `duckdb.sql(...)` sin conexión usa una base de datos en memoria, vacía, que DuckDB crea al vuelo. Para leer un fichero no hace falta más. Y la salida es la tabla de DuckDB de siempre, porque no se ha pedido `.df()`.

10. APIs

10.1 Qué es una API

Una **API web** es un servicio que responde con datos en lugar de con páginas. Le haces una petición a una dirección (una URL), con unos parámetros, y te devuelve normalmente un JSON. Es como pedir una consulta a una base de datos que está en otro sitio y que no es tuya.

Marejada usa APIs a diario sin saberlo: la pasarela de pago de la web, el seguimiento de los transportistas, el tipo de cambio del banco. Y en el trabajo de datos son una de las fuentes más habituales: traer datos de un CRM, de una herramienta de marketing o, como aquí, de un servicio público.

En este capítulo vas a usar **Frankfurter**, un servicio gratuito y sin registro que publica los tipos de cambio de referencia del Banco Central Europeo. Se usa con `requests`, el paquete que instalaste en el Volumen 05 (8.5).

PIEZA	QUÉ ES	EJEMPLO
URL base	La dirección del servicio	`https://api.frankfurter.dev/v1/`
Ruta (*endpoint*)	Qué pides	`latest` (el último tipo), `2025-01-10` (un día)
Parámetros	Filtros, detrás de un `?`	`?base=EUR&symbols=GBP`
Código de estado	Un número que dice cómo ha ido	200 bien; 4xx error tuyo; 5xx error del servidor
Cuerpo	La respuesta	Un JSON

10.2 La primera petición

PYTHON · EN TU NOTEBOOK

```python
import requests

r = requests.get(
    "https://api.frankfurter.dev/v1/latest",
    params={"base": "EUR", "symbols": "GBP"},
    timeout=10,
)
print(r.status_code)
print(r.url)
r.json()
```

RESULTADO

```
200
https://api.frankfurter.dev/v1/latest?base=EUR&symbols=GBP
{'amount': 1.0, 'base': 'EUR', 'date': '2026-10-26', 'rates': {'GBP': 0.8382}}
```

DESMONTANDO EL COMANDO

`requests.get(url, params={...}, timeout=10)`

`requests.get`	Hace una petición GET: "dame esto". Es la que usarás casi siempre para leer.
`params={...}`	Los parámetros como diccionario. `requests` los añade a la URL con el formato correcto (`?base=EUR&symbols=GBP`), y se encarga de los caracteres especiales. Nunca los pegues a mano en la URL.
`timeout=10`	Cuánto esperar, en segundos, antes de rendirse. **Ponlo siempre**: sin él, si el servidor no contesta, tu programa se queda esperando para siempre.
`r.status_code`	200: todo bien.
`r.url`	La URL que se ha pedido de verdad. Muy útil para depurar.
`r.json()`	Convierte el cuerpo de la respuesta (un texto JSON) en un diccionario de Python.

La fecha de la respuesta es la del día en que se ejecutó esta celda para preparar el volumen. Cuando la ejecutes tú, saldrá otra fecha y otro tipo de cambio: es un dato vivo.

10.3 Cuando algo va mal

Una API no da excepciones de Python cuando algo falla: responde con un código de estado de error. Si no lo miras, sigues adelante con un JSON que no es el que esperabas. Una fecha que no existe:

PYTHON · EN TU NOTEBOOK

```
r = requests.get("https://api.frankfurter.dev/v1/2025-13-01", timeout=10)
print(r.status_code)
r.json()
```

RESULTADO

```
422
{'message': 'invalid date'}
```

Un 422 y un mensaje. Fíjate en que la línea de `r.json()` no ha fallado: el error ha llegado como datos. Lo seguro es convertir cualquier código de error en una excepción con `raise_for_status()`:

PYTHON · EN TU NOTEBOOK

```
r.raise_for_status()
```

ERROR

```
---------------------------------------------------------------------------
HTTPError                                 Traceback (most recent call last)
Cell In[42], line 1
----> 1 r.raise_for_status()

File /home/claude/nb/.venv/lib/python3.12/site-packages/requests/models.py:1167, in
Response.raise_for_status(self)
   1162     http_error_msg = (
   1163         f"{self.status_code} Server Error: {reason} for url: {self.url}"
   1164     )
   1166 if http_error_msg:
-> 1167     raise HTTPError(http_error_msg, response=self)

HTTPError: 422 Client Error: Unprocessable Entity for url: https://api.frankfurter.dev/v1/2025-13-01
```

CÓDIGO	SIGNIFICA	QUÉ HACER
200	Bien	Seguir
400, 422	La petición está mal (un parámetro, un formato)	Corregir la petición
401, 403	Sin permiso (falta la clave o no vale)	Revisar la clave de acceso
404	Eso no existe	Revisar la ruta
429	Demasiadas peticiones	Esperar y reintentar más despacio (10.6)
500, 502, 503	El servidor ha fallado	Reintentar más tarde; no es culpa tuya

Y luego están los errores en los que ni siquiera hay respuesta: el servidor no contesta o no hay red. Esos sí llegan como excepciones de Python. Con un `timeout` absurdamente corto se provoca uno:

PYTHON · EN TU NOTEBOOK

```
try:
    requests.get("https://api.frankfurter.dev/v1/latest", timeout=0.001)
except requests.exceptions.RequestException as e:
    print(type(e).__name__)
```

RESULTADO

```
ReadTimeout
```

`RequestException` es la madre de todos los errores de `requests` (el de tiempo agotado, el de conexión, el de `raise_for_status`), así que atraparla los cubre todos. Es el `try` / `except` del Volumen 05 (6.2), con el tipo concreto de esta librería.

10.4 Un rango de fechas, y su trampa

Frankfurter devuelve una serie entera si se le pide un rango, con dos fechas separadas por dos puntos:

PYTHON · EN TU NOTEBOOK

```
r = requests.get(
    "https://api.frankfurter.dev/v1/2025-01-01..2025-01-10",
    params={"base": "EUR", "symbols": "GBP"},
    timeout=10,
)
r.raise_for_status()
datos = r.json()
print(datos["start_date"], datos["end_date"])
datos["rates"]
```

RESULTADO

```
2024-12-31 2025-01-10
{'2024-12-31': {'GBP': 0.82918},
 '2025-01-02': {'GBP': 0.83118},
 '2025-01-03': {'GBP': 0.82993},
 '2025-01-06': {'GBP': 0.83098},
 '2025-01-07': {'GBP': 0.82915},
 '2025-01-08': {'GBP': 0.83403},
 '2025-01-09': {'GBP': 0.83808},
 '2025-01-10': {'GBP': 0.83693}}
```

Dos detalles que pasarían desapercibidos. El primero: has pedido desde el 1 de enero y la serie **empieza el 31 de diciembre**. El 1 de enero es festivo y el Banco Central Europeo no publica tipo, así que el servicio te da el último disponible antes de esa fecha. El segundo: no hay datos de los fines de semana (el 4 y el 5 de enero no están). Si haces una media "de enero" con esta respuesta tal cual, incluirás un día de diciembre. Hay que filtrar:

PYTHON · EN TU NOTEBOOK

```
gbp = pd.Series({fecha: tipos["GBP"] for fecha, tipos in datos["rates"].items()})
gbp.index = pd.to_datetime(gbp.index)
enero = gbp[gbp.index >= "2025-01-01"]
print(len(gbp), len(enero))
enero.mean().round(5)
```

RESULTADO

```
8 7
np.float64(0.8329)
```

CUIDADO

Lee siempre lo que la API te ha devuelto, no lo que le has pedido. Fechas de inicio que se mueven, días que faltan, monedas que no existen y que se ignoran sin error: los servicios tienen sus reglas, y la documentación no siempre las cuenta.

10.5 Pedir por tramos

Muchas APIs no devuelven todo de una vez: dan los resultados por **páginas** (los 100 primeros, y un enlace o un número para pedir los siguientes), o limitan el periodo de cada petición. Frankfurter no pagina, pero pedirle varios años de golpe es mala educación con un servicio gratuito, y además una respuesta enorme es más frágil. El patrón es el mismo en los dos casos: un bucle que pide un tramo, lo guarda y pasa al siguiente.

PYTHON · EN TU NOTEBOOK

```python
import time


def tipos_de_cambio(anio, moneda="GBP"):
    """Serie diaria EUR -> moneda de un año completo, solo con fechas de ese año."""
    r = requests.get(
        f"https://api.frankfurter.dev/v1/{anio}-01-01..{anio}-12-31",
        params={"base": "EUR", "symbols": moneda},
        timeout=10,
    )
    r.raise_for_status()
    serie = pd.Series({f: t[moneda] for f, t in r.json()["rates"].items()}, name=moneda)
    serie.index = pd.to_datetime(serie.index)
    return serie[serie.index.year == anio]


tramos = []
for anio in [2024, 2025]:
    tramos.append(tipos_de_cambio(anio))
    time.sleep(1)

gbp = pd.concat(tramos)
print(len(gbp), gbp.index.min().date(), gbp.index.max().date())
```

RESULTADO

```
511 2024-01-02 2025-12-31
```

DESMONTANDO EL COMANDO

`for anio in [...]: tramos.append(...); time.sleep(1)`

`tipos_de_cambio(anio)`	Una función por tramo: pide, comprueba el código, convierte y filtra. Si algo falla, `raise_for_status` para el bucle con un error claro, en lugar de seguir con un tramo vacío.
`serie[serie.index.year == anio]`	El filtro de la trampa del 10.4: fuera el 31 de diciembre del año anterior.
`time.sleep(1)`	Espera un segundo entre peticiones. Es la cortesía básica con cualquier API, y la mejor forma de no recibir un 429.
`pd.concat(tramos)`	Junta los tramos en una sola serie (7.3).

Con una API paginada el bucle sería un `while` que sigue mientras la respuesta traiga una página siguiente; la estructura (pedir, comprobar, guardar, esperar) es la misma.

10.6 Límites y caché

Los servicios gratuitos limitan cuántas peticiones aceptan por minuto o por día. Al preparar este volumen, otro servicio público (uno de datos meteorológicos) contestó con un **429** porque se había pasado su límite diario: no es teoría. Dos reglas:

- **No pidas dos veces lo mismo.** Los tipos de cambio de 2025 no van a cambiar. Se piden una vez, se guardan en un fichero, y las siguientes veces se lee el fichero. Eso se llama una **caché**.
- **Si recibes un 429, espera.** Muchas APIs dicen cuánto en la cabecera `Retry-After` de la respuesta (`r.headers["Retry-After"]`).

Una caché sencilla, con un CSV:

PYTHON · EN TU NOTEBOOK

```python
def tipos_con_cache(anio, moneda="GBP"):
    fichero = carpeta / f"tipos_{moneda}_{anio}.csv"
    if fichero.exists():
        print("desde la caché:", fichero.name)
        return pd.read_csv(fichero, index_col=0, parse_dates=True).squeeze("columns")
    print("pidiendo a la API:", anio)
    serie = tipos_de_cambio(anio, moneda)
    serie.to_csv(fichero)
    return serie


a = tipos_con_cache(2025)
b = tipos_con_cache(2025)
print(len(a), len(b), (a == b).all())
```

RESULTADO

```
pidiendo a la API: 2025
desde la caché: tipos_GBP_2025.csv
255 255 True
```

La primera vez llama a la API y guarda; la segunda, lee el fichero. Las dos series son idénticas.

DESMONTANDO EL COMANDO

`pd.read_csv(fichero, index_col=0, parse_dates=True).squeeze("columns")`

`index_col=0`	La primera columna del CSV (las fechas) se usa como índice.
`parse_dates=True`	Convierte ese índice en fechas.
`.squeeze("columns")`	El CSV se lee como un DataFrame de una sola columna; `squeeze` lo convierte en una Series, que es lo que devolvía la otra rama de la función. Las dos ramas devuelven lo mismo.

LAS CLAVES, FUERA DEL CÓDIGO

Frankfurter no pide clave, pero casi todas las APIs sí (una cadena secreta que te identifica). Una clave **nunca** se escribe en un notebook ni en un script, y menos en un repositorio público como el tuyo: hay programas que recorren GitHub buscándolas. Se guarda en una variable de entorno o en un fichero `.env` excluido en `.gitignore`, y se lee con `os.environ["NOMBRE"]`. Lo harás así en el Bloque 4 y en el 7.

11. Gráficos con matplotlib

11.1 Lo justo para explorar

Los informes de verdad los harás con Power BI a partir del Volumen 07. Pero mientras exploras unos datos, un gráfico rápido en el notebook te enseña en dos segundos lo que una tabla no: una tendencia, un mes raro, un salto. Para eso está **matplotlib**, que instalaste en el Volumen 01.

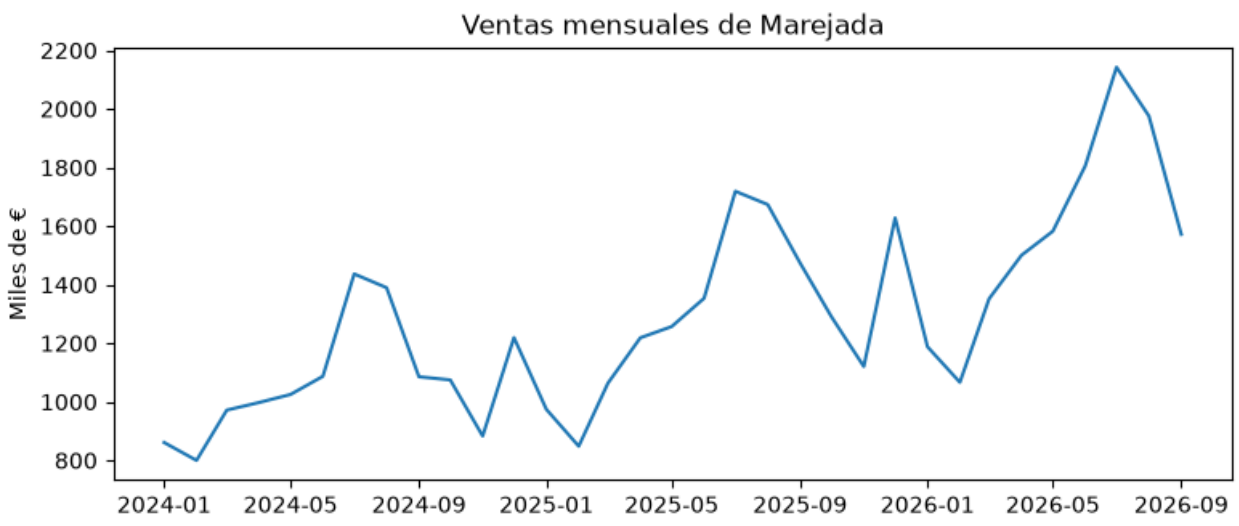
La forma recomendada de usarlo tiene siempre tres pasos: crear la figura y los ejes, dibujar en los ejes, y mostrar o guardar.

PYTHON · EN TU NOTEBOOK

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 3.5))
ax.plot(mensual.index, mensual / 1000)
ax.set_title("Ventas mensuales de Marejada")
ax.set_ylabel("Miles de €")
plt.show()
```

FIGURA · EN TU NOTEBOOK



DESMONTANDO EL COMANDO

`fig, ax = plt.subplots(figsize=(9, 3.5))`

`plt.subplots(...)`	Crea una figura (`fig`, el lienzo entero) y unos ejes (`ax`, la zona donde se dibuja). Casi todo se hace sobre `ax`.
`figsize=(9, 3.5)`	Ancho y alto en pulgadas.
`ax.plot(x, y)`	Una línea. `mensual` es la Series del 8.2, con los meses como índice.
`mensual / 1000`	Dibujar en miles hace el eje legible. Operación vectorizada (2.4).
`ax.set_title, ax.set_ylabel`	Título y nombre del eje.
`plt.show()`	Muestra la figura en el notebook.

En un segundo se ve lo que en el Volumen 03 costó varias consultas: el crecimiento, el pico de cada verano en julio, el de diciembre, el bajón de enero… y que el último punto, septiembre de 2026, baja. ¿Es real? Ya lo sabes del 8.2: es un mes completo (la fecha de corte es el 30), y sube un 6,6 % sobre el septiembre anterior. El gráfico hace las preguntas; los números las contestan.

11.2 Varias series y un gráfico de barras

Las ventas por canal, una línea por canal. Con la tabla en formato ancho (una columna por canal, 6.2), pandas dibuja cada columna:

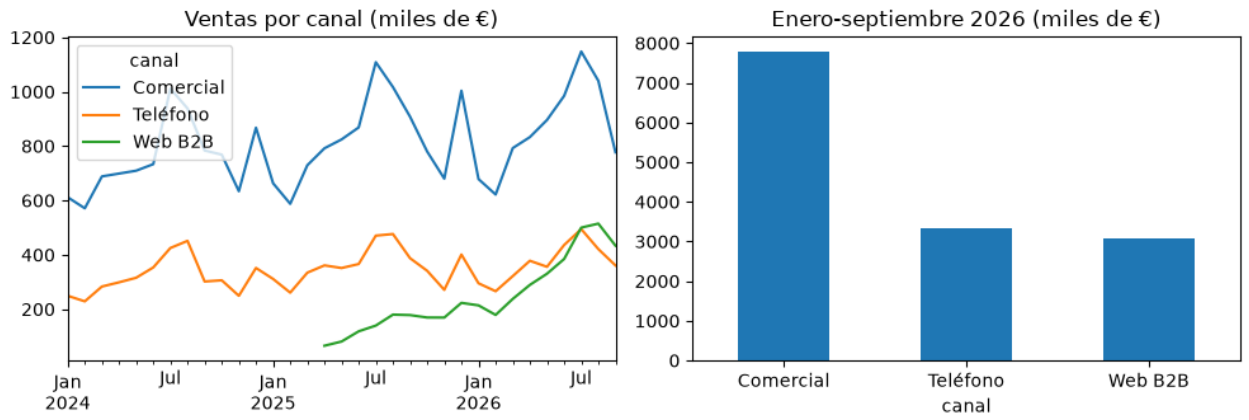
PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
por_canal = con.sql("""
    SELECT d.inicio_mes AS mes, f.canal, sum(f.importe_neto) / 1000 AS miles
    FROM gold.fact_ventas AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
    GROUP BY ALL
""").df()
con.close()

ancho = por_canal.pivot_table(index="mes", columns="canal", values="miles")

fig, (izq, der) = plt.subplots(1, 2, figsize=(10, 3.5))
ancho.plot(ax=izq, title="Ventas por canal (miles de €)")
ancho.loc["2026"].sum().plot.bar(ax=der, title="Enero-septiembre 2026 (miles de €)", rot=0)
izq.set_xlabel("")
plt.tight_layout()
plt.show()
```

FIGURA · EN TU NOTEBOOK



DESMONTANDO EL COMANDO

`fig, (izq, der) = plt.subplots(1, 2, ...)`

`plt.subplots(1, 2)`	Una fila y dos columnas de ejes: dos gráficos lado a lado. Devuelve los dos ejes, que se desempaquetan en `izq` y `der` (Volumen 05, 5.3).
`ancho.plot(ax=izq, ...)`	pandas dibuja una línea por columna, con su leyenda, en los ejes que le digas.
`ancho.loc["2026"]`	Con fechas en el índice, `.loc` acepta un año como texto: todas las filas de 2026.
`.sum().plot.bar(...)`	Suma cada canal y lo dibuja como barras. `rot=0` deja las etiquetas horizontales.
`plt.tight_layout()`	Ajusta los márgenes para que no se pisen los títulos.

11.3 Guardar la imagen

Para mandar un gráfico a alguien, se guarda con `savefig` **antes** de `plt.show()` (después de mostrarla, la figura se vacía):

PYTHON · EN TU NOTEBOOK

```
fig, ax = plt.subplots(figsize=(9, 3.5))
ax.plot(mensual.index, mensual / 1000, color="#1f4e79")
ax.set_title("Ventas mensuales de Marejada (miles de €)")
ax.grid(alpha=0.3)
fig.savefig("../entregables/ventas_mensuales.png", dpi=150, bbox_inches="tight")
plt.close(fig)
Path("../entregables/ventas_mensuales.png").exists()
```

RESULTADO

```
True
```

`dpi=150` da una imagen nítida; `bbox_inches="tight"` recorta los márgenes blancos; y `plt.close(fig)` cierra la figura sin mostrarla, para que no se acumulen en memoria si generas muchas.

12. pandas y DuckDB, juntos

12.1 SQL sobre un DataFrame

DuckDB puede consultar un DataFrame de pandas **por su nombre de variable**, sin cargarlo en ninguna base:

PYTHON · EN TU NOTEBOOK

```
duckdb.sql("""
    SELECT status, count(*) AS pedidos
    FROM pedidos_web
    GROUP BY status
    ORDER BY pedidos DESC
""").df()
```

RESULTADO

```
       status  pedidos
0   completed     7543
1   cancelled      297
2     shipped       49
3  processing       36
4  sync_error       31
```

`pedidos_web` es el DataFrame del 3.4. DuckDB lo encuentra entre las variables del notebook y lo lee directamente, sin copiarlo. Y con `.df()` el resultado vuelve a ser un DataFrame. Las dos herramientas se pasan los datos en las dos direcciones, y eso cambia la pregunta de "¿pandas o SQL?" a "¿qué paso hago con cada una?".

12.2 Lo mismo, de las dos formas

Ventas por categoría en 2026, en pandas y en SQL, partiendo de los mismos DataFrames:

PYTHON · EN TU NOTEBOOK

```
fv = leido                      # gold.fact_ventas, leída del Parquet (9.2)
dp = productos                  # gold.dim_producto (2.1)

en_pandas = (
    fv[fv["fecha_key"] >= 20260101]
    .merge(dp[["producto_key", "categoria"]], on="producto_key", validate="many_to_one")
    .groupby("categoria")["importe_neto"].sum()
    .sort_values(ascending=False)
    .round(2)
)

en_sql = duckdb.sql("""
    SELECT p.categoria, round(sum(f.importe_neto), 2) AS importe_neto
    FROM fv AS f
    JOIN dp AS p USING (producto_key)
    WHERE f.fecha_key >= 20260101
    GROUP BY p.categoria
    ORDER BY importe_neto DESC
""").df().set_index("categoria")["importe_neto"]

print(en_pandas.head(3))
(en_pandas - en_sql).abs().max()
```

RESULTADO

```
categoria
Vinos           3567168.62
Cervezas        2855036.02
Alimentación    2161178.51
Name: importe_neto, dtype: float64
np.float64(0.0)
```

El mismo resultado (la diferencia máxima es cero o una fracción de céntimo de redondeo). ¿Y la velocidad? Con `%timeit`, una orden especial de los notebooks que repite una línea varias veces y mide:

PYTHON · EN TU NOTEBOOK

```
%timeit -n 3 -r 3 fv.groupby("producto_key")["importe_neto"].sum()
%timeit -n 3 -r 3 duckdb.sql("SELECT producto_key, sum(importe_neto) FROM fv GROUP BY 1").df()
```

RESULTADO

```
6.77 ms ± 296 µs per loop (mean ± std. dev. of 3 runs, 3 loops each)
135 ms ± 17.9 ms per loop (mean ± std. dev. of 3 runs, 3 loops each)
```

Aquí gana pandas, y con claridad: unos 7 milisegundos contra unos 150 (en tu Mac las cifras serán otras, pero la proporción parecida). Tiene sentido: los datos ya están en la memoria de pandas, con su formato, y DuckDB tiene que leerlos del DataFrame, agrupar y construir otro DataFrame para devolverlo. Cuando los datos están en la base o en un Parquet y son mucho más grandes, la balanza se invierte. Y para una persona, 150 milisegundos siguen siendo instantáneos: a este tamaño, la velocidad no decide. Lo que decide es dónde están los datos y qué vas a hacer después.

12.3 Cuándo cada uno

SITUACIÓN	MEJOR CON	POR QUÉ
Agregar, unir y filtrar tablas grandes que ya están en una base	DuckDB	Es su trabajo, y no carga todo en memoria

SITUACIÓN	MEJOR CON	POR QUÉ
El resultado va a Power BI, a otra tabla o a otra persona que sabe SQL	DuckDB (SQL)	El SQL lo lee cualquiera y se lleva a Fabric o a Databricks casi tal cual
Limpiar un fichero raro (JSON anidado, Excel con títulos)	pandas	`json_normalize`, `read_excel` y compañía
Llamar a una API y convertir la respuesta	pandas	Los datos nacen en Python
Un gráfico, un Excel para alguien, un modelo de machine learning	pandas	Es lo que esas librerías esperan
Un cálculo fila a fila con lógica de Python	pandas	Pero intenta que sea vectorizado
Más de unos pocos millones de filas en tu Mac de 8 GB	DuckDB	pandas necesita que todo quepa en memoria, y suele ocupar varias veces el tamaño del fichero

IDEA CLAVE

El flujo típico en el trabajo: **DuckDB (o el SQL de la plataforma) hace el trabajo pesado y entrega una tabla pequeña; pandas la termina** (un formato, un gráfico, un Excel, un modelo). Si te encuentras haciendo en pandas un `merge` de cuatro tablas de millones de filas, probablemente ese paso era de SQL.

PARA EL EXAMEN

En el DP-600 y en el examen de Databricks te preguntarán por notebooks que mezclan SQL y Python (PySpark), y por el formato de los ficheros del lakehouse: Parquet y Delta. El concepto es el de este capítulo: el DataFrame es la tabla de Python, y SQL y Python se pasan los datos dentro del mismo notebook.

TICKETS

Tickets del Volumen 06

Seis encargos con pandas, Parquet, una API y gráficos. Entre ellos, el cruce de la web con el ERP que quedó pendiente.

Tickets · Volumen 06

Soluciones · Volumen 06

Tickets · Volumen 06

Los notebooks van a `notebooks/` con el nombre del ticket (`mjd-601_web_erp.ipynb`), y lo que se entrega a otra persona, a `entregables/`. La fecha de corte, el 30 de septiembre de 2026.

TICKET MJD-601 De: Paula Herrero · Marketing y tienda online

Prioridad: alta

En el MJD-505 encontrasteis los pedidos web que no llegaron al ERP. Ahora quiero el cuadre completo: pedido a pedido, ¿la tienda web y el ERP dicen lo mismo? Mismos pedidos, mismo estado, mismo importe.

CRITERIOS DE ACEPTACIÓN

- Todos los pedidos de `pedidos_web.json` cruzados con los del ERP, en las dos direcciones.
- Los estados de la web traducidos a los del ERP, y cuántos no coinciden.
- Los pedidos con importe distinto, cuántos son y por qué.

TICKET MJD-602 De: Lucía Gómez · Administración y finanzas

Prioridad: alta

En el MJD-204 cuadrasteis marzo de 2025 con mi hoja de facturación. Quiero lo mismo para **todos los meses**, de enero de 2024 a septiembre de 2026. Si alguno no cuadra, quiero saber exactamente qué pedidos lo explican.

CRITERIOS DE ACEPTACIÓN

- La hoja `facturacion_mensual_2024-2026.xlsx` leída con pandas y comparada mes a mes con gold.
- Lista de los meses que no cuadran, con la diferencia.
- Los pedidos que explican cada diferencia.

TICKET MJD-603 De: Elena Pérez · Responsable de datos

Prioridad: media

Para el Bloque 2 vamos a subir gold a Fabric, y allí se trabaja con Parquet. Exporta las siete... no, perdón: las **seis** tablas de gold a Parquet y demuéstrame que no se ha perdido nada por el camino.

CRITERIOS DE ACEPTACIÓN

- Un fichero Parquet por tabla de gold en `datos/exportaciones/gold/`.
- Comprobación de filas y de una suma de control por tabla, comparando la base con el fichero.

TICKET MJD-604 De: Carmen Ruiz · Dirección general

Prioridad: media

Vamos a negociar con Old Pier, el proveedor escocés del whisky, que factura en libras. Necesito la media mensual del cambio euro-libra de 2025, y cuánto nos habría costado un pedido fijo de 10.000 £ cada mes al cambio medio de ese mes.

ENTREGABLE

`entregables/mjd-604_cambio_gbp_2025.csv`, con el mes, el cambio medio y el coste en euros.

TICKET MJD-605 De: Íñigo Sainz · Director comercial

Prioridad: media

Para la reunión del comité: un gráfico de las ventas mensuales por canal, de 2024 a hoy, que se entienda sin explicarlo. En imagen, que lo pego en la presentación.

ENTREGABLE

`entregables/mjd-605_ventas_canal.png`.

TICKET MJD-606 De: Raquel Fernández · Jefa de ventas Cantabria

Prioridad: media

El ranking de mis comerciales de Cantabria, enero a septiembre de 2026 contra el mismo periodo de 2025, en un Excel que pueda abrir y reenviar. Con los números como los escribimos aquí, no como en inglés.

ENTREGABLE

`entregables/mjd-606_ranking_cantabria.xlsx`, y el mismo ranking en el notebook con formato español.

Soluciones · Volumen 06

MJD-601 · La tienda web contra el ERP

Tres preguntas, tres cruces. Primero, preparar las dos tablas: los pedidos de la web aplanados (3.4) con su importe calculado a partir de las líneas, y los pedidos del ERP desde silver.

PYTHON · EN TU NOTEBOOK

```
import json
import duckdb
import pandas as pd

with open("../datos/web/pedidos_web.json", encoding="utf-8") as f:
    web = json.load(f)

cab = pd.json_normalize(web["orders"]).drop(columns="items")
lin = pd.json_normalize(web["orders"], record_path="items", meta=["order_id"])
lin["importe"] = lin["qty"] * lin["unit_price"] * (1 - lin["discount"] / 100)
cab = cab.merge(
    lin.groupby("order_id", as_index=False)["importe"].sum(),
    on="order_id", how="left", validate="one_to_one",
)
cab["erp_pedido_id"] = cab["erp_pedido_id"].astype("Int64")

con = duckdb.connect("../db/marejada.duckdb", read_only=True)
erp = con.sql("""
    SELECT p.pedido_id, p.estado, coalesce(sum(l.importe_neto), 0) AS importe_erp
    FROM silver.pedidos AS p
    LEFT JOIN silver.lineas_pedido AS l USING (pedido_id)
    WHERE p.canal = 'Web B2B'
    GROUP BY p.pedido_id, p.estado
""").df()
con.close()

print(cab.shape, erp.shape)
print(cab["erp_pedido_id"].isna().sum())
```

RESULTADO

```
(7956, 8) (7925, 3)
31
```

Pedidos en las dos direcciones. Un `merge` completo con `indicator` (7.2). Lo natural sería `validate="one_to_one"`, porque cada pedido web es un pedido del ERP; pero los 31 pedidos sin `erp_pedido_id` tienen todos la misma clave, el nulo, y pandas los vería como repetidos. Así que la unicidad de la web se comprueba aparte, sin los nulos, y el `merge` valida solo el lado del ERP:

PYTHON · EN TU NOTEBOOK

```
assert cab["erp_pedido_id"].dropna().is_unique, "un pedido del ERP aparece dos veces en la web"

cruce = cab.merge(
    erp, left_on="erp_pedido_id", right_on="pedido_id",
    how="outer", indicator=True, validate="many_to_one",
)
print(cruce["_merge"].value_counts())
cruce.loc[cruce["_merge"] == "left_only", "status"].value_counts()
```

RESULTADO

```
_merge
both          7925
left_only       31
right_only       0
Name: count, dtype: int64
status
sync_error    31
Name: count, dtype: int64
```

Estados. La tabla de traducción como diccionario, y la comparación:

PYTHON · EN TU NOTEBOOK

```
traduccion = {"completed": ["Entregado", "Devuelto parcial"], "cancelled": ["Cancelado"],
              "processing": ["Pendiente"], "shipped": ["En reparto"]}
ambos = cruce[cruce["_merge"] == "both"].copy()
ambos["estado_ok"] = [e in traduccion.get(s, []) for s, e in zip(ambos["status"], ambos["estado"])]
print(ambos["estado_ok"].value_counts())
pd.crosstab(ambos["status"], ambos["estado"])
```

RESULTADO

```
estado_ok
True    7925
Name: count, dtype: int64
estado      Cancelado  Devuelto parcial  En reparto  Entregado  Pendiente
status
cancelled         297                 0           0          0          0
completed           0               107           0       7436          0
processing          0                 0           0          0         36
shipped             0                 0          49          0          0
```

Importes. Con una tolerancia de un céntimo, porque se comparan decimales calculados de dos formas:

PYTHON · EN TU NOTEBOOK

```
ambos["dif"] = (ambos["importe"] - ambos["importe_erp"]).round(2)
distintos = ambos[ambos["dif"].abs() > 0.01]
print(len(distintos))
print(distintos["estado"].value_counts())
print(distintos["dif"].describe().round(2))
```

RESULTADO

```
107
estado
Devuelto parcial    107
Name: count, dtype: int64
count    107.00
mean      63.64
std       78.89
min        4.79
25%       19.18
50%       37.07
75%       75.65
max      475.61
Name: dif, dtype: float64
```

Los pedidos con importe distinto son exactamente los «Devuelto parcial», y en todos la web dice **más** que el ERP. La explicación está en las líneas: el ERP registra la devolución como líneas negativas dentro del mismo pedido (Volumen 02), y la web no. Segunda vía de comprobación: si se quitan las líneas negativas del ERP, los importes deberían cuadrar.

PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
sin_devol = con.sql("""
    SELECT pedido_id, sum(importe_neto) AS importe_sin_devoluciones
    FROM silver.lineas_pedido
    WHERE cantidad > 0
    GROUP BY pedido_id
""").df()
con.close()

check = distintos.merge(sin_devol, on="pedido_id", validate="one_to_one")
((check["importe"] - check["importe_sin_devoluciones"]).abs() > 0.01).sum()
```

RESULTADO

```
np.int64(0)
```

Cero: sin las devoluciones, los «Devuelto parcial» cuadran al céntimo.

REVISIÓN DE ELENA

El cuadre está completo y bien comprobado: 7.925 pedidos en los dos sistemas, ninguno en el ERP que no esté en la web, los 31 `sync_error` que ya conocíamos, todos los estados coherentes, y los importes que difieren explicados al céntimo por una segunda vía. Me gusta especialmente el `assert` con `is_unique` antes del cruce: si un día la web mandara un pedido dos veces, este notebook lo diría en lugar de duplicar importes.

Para Paula, cuéntalo en su idioma: «la web y el ERP coinciden en todo salvo en dos cosas: 31 pedidos que no llegaron (el MJD-505) y que la web no se entera de las devoluciones, así que en el panel de la tienda los pedidos con devolución aparecen por su importe original». Lo segundo es una petición para el proveedor de la web, no un error de datos. Y una pregunta de entrevista que te podrían hacer con esto: «¿cómo automatizarías este cuadre?». Respuesta: como una comprobación más, igual que las de silver, que se ejecuta cada noche y avisa si aparecen pedidos `left_only` nuevos.

MJD-602 · Contabilidad mes a mes

Leer la hoja (3.3), pasar el nombre del mes a número y totalizar por mes. Antes, una línea que cambia cómo enseña pandas los decimales en este notebook, para que no salgan en notación científica (`1.433232e+06`) ni con todos sus decimales:

PYTHON · EN TU NOTEBOOK

```
pd.set_option("display.float_format", "{:,.2f}".format)

conta = pd.read_excel("../datos/contabilidad/facturacion_mensual_2024-2026.xlsx", skiprows=3)
meses = ["enero", "febrero", "marzo", "abril", "mayo", "junio", "julio",
         "agosto", "septiembre", "octubre", "noviembre", "diciembre"]
conta["mes"] = conta["Mes"].map({m: i for i, m in enumerate(meses, start=1)})
assert conta["mes"].notna().all(), "hay algún mes mal escrito"

contab = (
    conta.groupby(["Año", "mes"])["Base imponible (€)"].sum()
    .rename_axis(["anio", "mes"]).rename("contabilidad")
)
print(len(contab))
contab.head(3)
```

RESULTADO

```
33
anio  mes
2024  1     811,290.05
      2     805,880.38
      3     949,850.27
Name: contabilidad, dtype: float64
```

Gold, con el criterio del puente del Volumen 02: pedidos entregados o con devolución parcial, por mes de entrega, con los portes. Es la comprobación 7 de `04_comprobar_gold.sql`, ahora para todos los meses:

PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
gold = con.sql("""
    SELECT d.anio, d.mes, sum(f.importe_neto + f.gastos_envio) AS gold
    FROM gold.fact_pedidos AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_entrega_key
    WHERE f.estado IN ('Entregado', 'Devuelto parcial')
    GROUP BY d.anio, d.mes
""").df().set_index(["anio", "mes"])["gold"]

cuadre = pd.concat([contab, gold], axis=1)
cuadre["dif"] = (cuadre["contabilidad"] - cuadre["gold"]).round(2)
no_cuadran = cuadre[cuadre["dif"].abs() > 0.01]
print(len(cuadre), len(no_cuadran))
no_cuadran
```

RESULTADO

```
33 8
          contabilidad         gold      dif
anio mes
2024 7    1,433,547.21 1,433,231.86   315.35
     8    1,394,522.90 1,394,451.72    71.18
     10   1,062,285.93 1,062,172.99   112.94
     12   1,224,133.48 1,223,978.80   154.68
2025 1      991,585.53   991,137.19   448.34
     5    1,235,046.85 1,231,799.73 3,247.12
     7    1,676,382.85 1,675,403.02   979.83
     8    1,651,705.30 1,651,315.28   390.02
```

Ocho meses no cuadran, todos por poco, y siempre con contabilidad **por encima** de gold. Algo que contabilidad ha facturado en ese mes y gold no tiene en ese mes. ¿Qué pedidos entregados no tienen mes de entrega en gold? Los de la fecha imposible, que silver dejó a `NULL` (Volumen 02, MJD-210). Su fecha original está en bronze:

PYTHON · EN TU NOTEBOOK

```
raras = con.sql("""
    SELECT f.pedido_id, b.fecha_entrega AS fecha_en_erp,
           round(f.importe_neto + f.gastos_envio, 2) AS importe
    FROM gold.fact_pedidos AS f
    JOIN bronze.pedidos AS b USING (pedido_id)
    WHERE f.estado IN ('Entregado', 'Devuelto parcial')
      AND f.fecha_entrega_key IS NULL
""").df()
con.close()

raras["fecha_en_erp"] = pd.to_datetime(raras["fecha_en_erp"])
raras["anio"] = raras["fecha_en_erp"].dt.year
raras["mes"] = raras["fecha_en_erp"].dt.month
print(len(raras))
raras["anio"].value_counts().sort_index()
```

RESULTADO

```
23
anio
1900    9
2023    4
2024    4
2025    6
Name: count, dtype: int64
```

Los 23 pedidos de fecha imposible: 9 con el 1 de enero de 1900 y 14 con el año equivocado. Si contabilidad facturó **con la fecha que tenía el ERP**, los que caen dentro del periodo de la hoja (2024 a 2026) explicarían las diferencias. Comprobación: sumarlos por ese mes y compararlo con la diferencia.

PYTHON · EN TU NOTEBOOK

```
explican = raras[raras["anio"].between(2024, 2026)].groupby(["anio", "mes"])["importe"].sum()
comparacion = pd.concat([no_cuadran["dif"], explican.rename("pedidos_raros")], axis=1)
comparacion["resto"] = (comparacion["dif"] - comparacion["pedidos_raros"]).round(2)
comparacion
```

RESULTADO

```
              dif  pedidos_raros  resto
anio mes
2024 7     315.35         315.34   0.01
     8      71.18          71.18   0.00
     10    112.94         112.94   0.00
     12    154.68         154.67   0.01
2025 1     448.34         448.34   0.00
     5   3,247.12       3,247.13  -0.01
     7     979.83         979.83   0.00
     8     390.02         390.03  -0.01
```

Cuadra: cada diferencia es exactamente la suma de los pedidos que el ERP tenía fechados en ese mes, con algún céntimo de redondeo (las diferencias se redondearon mes a mes; los importes, pedido a pedido). Contabilidad facturó con la fecha errónea, un año antes de la entrega real.

PYTHON · EN TU NOTEBOOK

```
lista = raras[raras["anio"].between(2024, 2026)].sort_values("fecha_en_erp")
print(len(lista), round(lista["importe"].sum(), 2))
lista[["pedido_id", "fecha_en_erp", "importe"]]
```

RESULTADO

```
10 5719.46
     pedido_id fecha_en_erp  importe
6   2025013364   2024-07-15   315.34
9   2025015502   2024-08-07    71.18
10  2025021740   2024-10-13   112.94
11  2025026215   2024-12-10   154.67
12  2026000597   2025-01-12   448.34
15  2026009133   2025-05-04   534.02
16  2026010798   2025-05-21 2,713.11
18  2026015558   2025-07-07   485.69
20  2026018492   2025-07-31   494.14
21  2026018807   2025-08-05   390.03
```

REVISIÓN DE ELENA

Muy buen trabajo de detective: de «ocho meses no cuadran» a «estos diez pedidos, con esta fecha, por este importe», y comprobado. Lo de que contabilidad facturase con el año equivocado es grave para Lucía, porque esos pedidos están en el periodo fiscal que no les toca: un pedido entregado en julio de 2026 está facturado en julio de 2025.

Dos cosas para el informe. La primera: dile que son diez pedidos, no ocho meses; y añade los otros 13, que no podemos comprobar con su hoja porque caen fuera de ella. Hay cuatro fechados en 2023: si se facturaron con esa fecha, están en un ejercicio cerrado, y eso es lo más delicado. Y nueve con 1900, que no están en ninguna fila de la hoja: o están sin facturar o se facturaron por otra vía. Eso solo lo puede mirar Lucía en su sistema. La segunda: la causa está en el ERP, no en contabilidad. Que lo sepa Marcos, porque la fecha de entrega la meten los repartidores.

MJD-603 · Gold en Parquet

Un bucle por las tablas de gold. La lista se pide a la propia base, para no olvidar ninguna:

PYTHON · EN TU NOTEBOOK

```
from pathlib import Path

destino = Path("../datos/exportaciones/gold")
destino.mkdir(parents=True, exist_ok=True)

con = duckdb.connect("../db/marejada.duckdb", read_only=True)
tablas = [t for (t,) in con.sql("""
    SELECT table_name FROM information_schema.tables
    WHERE table_schema = 'gold' ORDER BY table_name
""").fetchall()]
print(tablas)

for t in tablas:
    con.sql(f"SELECT * FROM gold.{t}").df().to_parquet(destino / f"{t}.parquet", index=False)
con.close()

sorted(f.name for f in destino.glob("*.parquet"))
```

RESULTADO

```
['dim_cliente', 'dim_comercial', 'dim_fecha', 'dim_producto', 'fact_pedidos', 'fact_ventas']
['dim_cliente.parquet',
 'dim_comercial.parquet',
 'dim_fecha.parquet',
 'dim_producto.parquet',
 'fact_pedidos.parquet',
 'fact_ventas.parquet']
```

Para la comprobación, la segunda vía es **leer los ficheros con otra herramienta**: DuckDB, sin pasar por pandas. Por cada tabla, las filas y una suma de control de la primera columna (la clave):

PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
filas = []
for t in tablas:
    clave = con.sql(f"SELECT * FROM gold.{t} LIMIT 0").columns[0]
    consulta = f"SELECT count(*), sum({clave}) FROM "
    base = con.sql(consulta + f"gold.{t}").fetchone()
    fichero = con.sql(consulta + f"'{destino / (t + '.parquet')}'").fetchone()
    filas.append((t, clave, *base, *fichero))
con.close()

control = pd.DataFrame(filas, columns=["tabla", "clave", "filas_base", "suma_base", "filas_parquet",
"suma_parquet"])
control["ok"] = (control["filas_base"] == control["filas_parquet"]) & (control["suma_base"] ==
control["suma_parquet"])
control
```

RESULTADO

```
          tabla          clave  filas_base        suma_base  filas_parquet  \
0    dim_cliente    cliente_key        1349           909225           1349
1  dim_comercial  comercial_key          15              104             15
2      dim_fecha      fecha_key        1096      22194722043           1096
3   dim_producto   producto_key         236            27729            236
4   fact_pedidos      pedido_id       77448  156834322518410          77448
5    fact_ventas      pedido_id      366928  743038294681089         366928

      suma_parquet    ok
0           909225  True
1              104  True
2      22194722043  True
3            27729  True
4  156834322518410  True
5  743038294681089  True
```

REVISIÓN DE ELENA

Bien: la lista de tablas sacada de la base y no escrita a mano, y la comprobación hecha con una herramienta distinta de la que escribió. Perdona la broma del enunciado; era para ver si contabas las tablas o te fiabas de mí. Hiciste bien.

Para la siguiente vez, una suma de la clave detecta filas que faltan o sobran, pero no un importe que se haya estropeado por el camino. En las tablas de hechos, añade también la suma de `importe_neto`. Y una cosa que te preguntarán en Fabric: ¿qué pasa con los `DECIMAL`? Abre `fact_pedidos.parquet` y mira el tipo de `gastos_envio`: pandas lo convirtió en `float64` al leerlo de la base (2.2), y así se ha guardado. Si el Parquet lo escribiera DuckDB directamente (`COPY gold.fact_pedidos TO 'x.parquet'`), el `DECIMAL` se conservaría. Para exportar desde una base, mejor el `COPY`.

MJD-604 · El cambio de la libra

La función con caché del 10.6, la serie de 2025, y la media por mes con `resample` (8.1). Primero se deshace el formato de dos decimales del MJD-602 (`reset_option`), porque aquí el tipo de cambio necesita cinco:

PYTHON · EN TU NOTEBOOK

```
import time
import requests

pd.reset_option("display.float_format")
carpeta = Path("../datos/exportaciones")


def tipos_de_cambio(anio, moneda="GBP"):
    r = requests.get(
        f"https://api.frankfurter.dev/v1/{anio}-01-01..{anio}-12-31",
        params={"base": "EUR", "symbols": moneda},
        timeout=10,
    )
    r.raise_for_status()
    serie = pd.Series({f: t[moneda] for f, t in r.json()["rates"].items()}, name=moneda)
    serie.index = pd.to_datetime(serie.index)
    return serie[serie.index.year == anio]


def tipos_con_cache(anio, moneda="GBP"):
    fichero = carpeta / f"tipos_{moneda}_{anio}.csv"
    if fichero.exists():
        return pd.read_csv(fichero, index_col=0, parse_dates=True).squeeze("columns")
    serie = tipos_de_cambio(anio, moneda)
    serie.to_csv(fichero)
    return serie


gbp = tipos_con_cache(2025)
print(len(gbp), gbp.index.min().date(), gbp.index.max().date())

tabla = gbp.resample("MS").agg(["mean", "count"])
tabla.columns = ["gbp_por_eur", "dias"]
tabla["coste_eur"] = 10_000 / tabla["gbp_por_eur"]
tabla.round({"gbp_por_eur": 5, "coste_eur": 2})
```

RESULTADO

```
255 2025-01-02 2025-12-31
            gbp_por_eur  dias  coste_eur
2025-01-01      0.83908    22   11917.80
2025-02-01      0.83071    20   12037.90
2025-03-01      0.83703    21   11947.07
2025-04-01      0.85379    20   11712.53
2025-05-01      0.84350    21   11855.43
2025-06-01      0.84981    21   11767.34
2025-07-01      0.86469    23   11564.88
2025-08-01      0.86528    21   11557.00
2025-09-01      0.86895    22   11508.20
2025-10-01      0.87155    23   11473.78
2025-11-01      0.87996    20   11364.09
2025-12-01      0.87500    21   11428.57
```

El tipo es «libras por euro» (un euro compra 0,8x libras), así que 10.000 £ cuestan `10000 / tipo` euros: cuanto más baja el tipo, más caro sale el whisky. Y el resumen para Carmen, más la segunda vía: la media anual de los días debe estar cerca de la media de las medias mensuales (no es idéntica, porque cada mes tiene distinto número de días):

PYTHON · EN TU NOTEBOOK

```
coste = tabla["coste_eur"]
print(f"Más barato: {coste.idxmin():%B} ({coste.min():,.2f} €)")
print(f"Más caro:   {coste.idxmax():%B} ({coste.max():,.2f} €)")
print(f"Total 2025: {coste.sum():,.2f} € por 120.000 £")
print(round(gbp.mean(), 5), round(tabla["gbp_por_eur"].mean(), 5))

tabla.reset_index(names="mes").to_csv("../entregables/mjd-604_cambio_gbp_2025.csv", index=False)
```

RESULTADO

```
Más barato: November (11,364.09 €)
Más caro:   February (12,037.90 €)
Total 2025: 140,134.60 € por 120.000 £
0.85679 0.85661
```

REVISIÓN DE ELENA

Correcto, y bien explicada la dirección del tipo, que es donde se equivoca casi todo el mundo la primera vez. La caché es lo que hay que hacer con datos que no cambian.

Dos detalles. Los meses salen en inglés (`%B` usa el idioma del sistema, que en Python por defecto es inglés): para Carmen, usa la lista de meses en español del MJD-602. Y una advertencia que debe ir en el correo: el tipo de referencia del BCE no es el que nos aplica el banco, que se queda un margen. Sirve para comparar meses entre sí y para negociar, no para cuadrar las facturas al céntimo.

MJD-605 · Ventas por canal, para el comité

Un gráfico para alguien que no lo va a estudiar: pocas líneas, ejes legibles, sin decoración, y el título diciendo lo que hay que ver.

PYTHON · EN TU NOTEBOOK

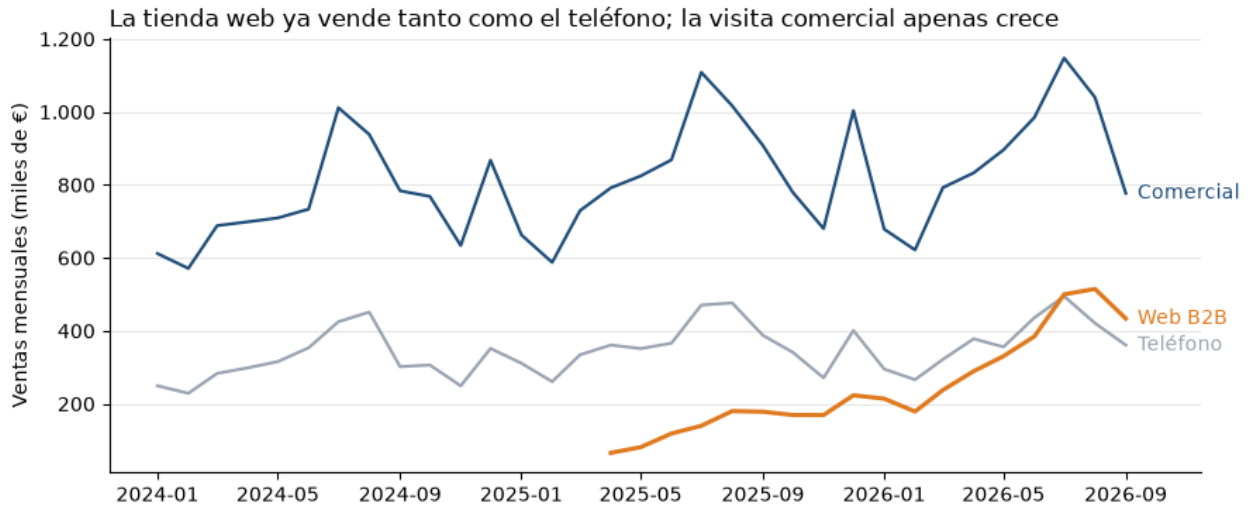
```
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

con = duckdb.connect("../db/marejada.duckdb", read_only=True)
canal = con.sql("""
    SELECT d.inicio_mes AS mes, f.canal, sum(f.importe_neto) / 1000 AS miles
    FROM gold.fact_ventas AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
    GROUP BY ALL
""").df().pivot_table(index="mes", columns="canal", values="miles")
con.close()

colores = {"Comercial": "#1f4e79", "Teléfono": "#9aa5b1", "Web B2B": "#e07a1f"}
fig, ax = plt.subplots(figsize=(10, 4))
for c in canal.columns:
    ax.plot(canal.index, canal[c], label=c, color=colores[c], linewidth=2.2 if c == "Web B2B" else 1.6)
    ax.annotate(c, (canal.index[-1], canal[c].iloc[-1]), xytext=(6, 0),
                textcoords="offset points", va="center", color=colores[c])

ax.set_title("La tienda web ya vende tanto como el teléfono; la visita comercial apenas crece",
             loc="left", fontsize=12)
ax.set_ylabel("Ventas mensuales (miles de €)")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda v, _: f"{v:,.0f}".replace(",", ".")))
ax.spines[["top", "right"]].set_visible(False)
ax.grid(axis="y", alpha=0.3)
ax.set_xlim(right=canal.index[-1] + pd.Timedelta(days=75))
fig.savefig("../entregables/mjd-605_ventas_canal.png", dpi=150, bbox_inches="tight")
plt.show()
```

FIGURA · EN TU NOTEBOOK



DESMONTANDO EL COMANDO

Lo que hace que se entienda sin explicarlo

`Un color por canal, y el protagonista destacado`	La web en naranja y más gruesa; el teléfono en gris. El ojo va donde está la historia.
`ax.annotate(...)`	El nombre de cada canal al final de su línea, en lugar de una leyenda en una esquina. No hay que ir y volver para saber qué línea es cuál.
`El título dice la conclusión`	No «Ventas por canal», sino lo que se ve. En un comité, la mitad de la gente solo lee el título.
`FuncFormatter(...)`	Los números del eje con punto de miles, como se escriben aquí. El `lambda` recibe cada valor y devuelve el texto.
`spines, grid`	Sin el marco superior y derecho, y una rejilla suave solo en horizontal: menos tinta que no dice nada.
`set_xlim(right=...)`	Deja sitio a la derecha para las etiquetas.

Antes de mandarlo, la segunda vía: ¿es verdad lo que dice el título? Enero a septiembre de cada año, en miles de euros, y la variación:

PYTHON · EN TU NOTEBOOK

```
ene_sep = canal[canal.index.month <= 9].groupby(canal.index[canal.index.month <= 9].year).sum()
print(ene_sep.round(0))
(ene_sep.pct_change() * 100).round(1)
```

RESULTADO

```
canal  Comercial  Teléfono  Web B2B
mes
2024      6746.0    2907.0      0.0
2025      7498.0    3319.0    763.0
2026      7771.0    3329.0   3083.0
canal  Comercial  Teléfono  Web B2B
mes
2024         NaN       NaN      NaN
2025        11.1      14.2      inf
2026         3.6       0.3    304.0
```

El título aguanta: de enero a septiembre, la visita comercial crece un 3,6 % y el teléfono un 0,3 %, mientras la web se multiplica por cuatro (+304 %; el `inf` de 2025 es una división entre el cero de 2024). Y en julio, agosto y septiembre de 2026 la web ya vendió más que el teléfono (500, 515 y 434 miles de euros, contra 495, 421 y 361).

REVISIÓN DE ELENA

Esto ya es un gráfico de comité. El título con la conclusión es lo que más diferencia un gráfico de analista de uno de un junior, y comprobar que el título es verdad antes de mandarlo, todavía más.

Un aviso para la presentación: la web arranca en abril de 2025, así que cualquiera que mire la línea naranja de 2024 verá que no existe. Si alguien pregunta «¿y por qué no estaba?», que Íñigo tenga la respuesta preparada. Y cuando lleguemos a Power BI, este mismo gráfico será interactivo y se actualizará solo; esta imagen es para hoy.

MJD-606 · Ranking de Cantabria, en Excel

Las ventas por comercial asignado al cliente, en los dos periodos (enero a septiembre), con la dimensión cliente de gold. La versión del cliente vigente en la fecha de cada venta ya está resuelta en la tabla de hechos (Volumen 04, SCD tipo 2):

PYTHON · EN TU NOTEBOOK

```
con = duckdb.connect("../db/marejada.duckdb", read_only=True)
ranking = con.sql("""
    SELECT c.comercial_asignado AS comercial,
           sum(f.importe_neto) FILTER (WHERE d.anio = 2026) AS ventas_2026,
           sum(f.importe_neto) FILTER (WHERE d.anio = 2025) AS ventas_2025
    FROM gold.fact_ventas AS f
    JOIN gold.dim_fecha AS d ON d.fecha_key = f.fecha_key
    JOIN gold.dim_cliente AS c ON c.cliente_key = f.cliente_key
    WHERE c.provincia = 'Cantabria' AND d.mes <= 9 AND d.anio IN (2025, 2026)
    GROUP BY c.comercial_asignado
""").df()
con.close()

ranking["variacion_pct"] = (ranking["ventas_2026"] / ranking["ventas_2025"] - 1) * 100
ranking = ranking.sort_values("ventas_2026", ascending=False).reset_index(drop=True)
ranking.index = ranking.index + 1
ranking
```

RESULTADO

	comercial	ventas_2026	ventas_2025	variacion_pct
1	Pablo Revuelta Gómez	1.818300e+06	1.642011e+06	10.736197
2	Raquel Fernández Cobo	1.708279e+06	1.662061e+06	2.780746
3	Daniel Gutiérrez Pérez	1.567632e+06	1.392262e+06	12.596014
4	Celia Gómez Sáiz	1.063105e+06	3.886159e+05	173.561933
5	Sara Ceballos Ortiz	1.045961e+06	8.477461e+05	23.381433
6	Lorena Arce Díaz	7.817259e+05	6.394953e+05	22.241080
7	Héctor Bustamante Lavín	NaN	5.964155e+05	NaN

Ahora, el formato español. Python escribe los números al estilo inglés (`1,234.5`). El truco habitual: formatear en inglés y luego intercambiar la coma y el punto, pasando por un carácter intermedio:

PYTHON · EN TU NOTEBOOK

```
def es(numero, decimales=2):
    if pd.isna(numero):
        return "—"
    texto = f"{numero:,.{decimales}f}"
    return texto.replace(",", "_").replace(".", ",").replace("_", ".")


print(es(1234567.891), es(-3.14159, 1), es(1000, 0), es(None))

vista = ranking.copy()
for col in ["ventas_2026", "ventas_2025"]:
    vista[col] = vista[col].map(lambda v: es(v) + " €" if pd.notna(v) else "—")
vista["variacion_pct"] = vista["variacion_pct"].map(lambda v: es(v, 1) + " %" if pd.notna(v) else "—")
vista
```

RESULTADO

```
1.234.567,89 -3,1 1.000 —
```

	comercial	ventas_2026	ventas_2025	variacion_pct
1	Pablo Revuelta Gómez	1.818.300,10 €	1.642.010,60 €	10,7 %
2	Raquel Fernández Cobo	1.708.278,51 €	1.662.060,82 €	2,8 %
3	Daniel Gutiérrez Pérez	1.567.632,07 €	1.392.262,49 €	12,6 %
4	Celia Gómez Sáiz	1.063.105,08 €	388.615,87 €	173,6 %
5	Sara Ceballos Ortiz	1.045.961,34 €	847.746,15 €	23,4 %
6	Lorena Arce Díaz	781.725,90 €	639.495,25 €	22,2 %
7	Héctor Bustamante Lavín	—	596.415,53 €	—

DESMONTANDO EL COMANDO

`f"{numero:,.{decimales}f}"` y los tres `replace`

`:,.2f`	Separador de miles (`,`) y dos decimales (`.2f`): el estilo inglés, `1,234,567.89`.
`{decimales}` dentro del formato	Las llaves anidadas permiten que el número de decimales sea una variable.
`replace(",", "_")`	Primero las comas pasan a un carácter que no aparece en un número…
`replace(".", ",")`	…después el punto decimal pasa a coma…
`replace("_", ".")`	…y por último el carácter intermedio pasa a punto. Sin el paso intermedio, el segundo `replace` también cambiaría las comas recién puestas.

CUIDADO

Esta columna formateada **ya no es un número**: es texto. Sirve para mostrar, no para calcular ni para un Excel. En un Excel, si escribes el texto `"1.234,56 €"`, Raquel no podrá sumar la columna. Al Excel van los **números**, y el formato se le pone al Excel.

Para el Excel, `to_excel` con los números de verdad, y el formato de las celdas puesto con `openpyxl` (el mismo paquete que lee los Excel, 3.3). Excel se encarga de mostrarlos con punto y coma según el idioma del ordenador de Raquel:

PYTHON · EN TU NOTEBOOK

```python
from openpyxl.styles import Font

ruta = "../entregables/mjd-606_ranking_cantabria.xlsx"
salida = ranking.rename(columns={
    "comercial": "Comercial", "ventas_2026": "Ventas ene-sep 2026 (€)",
    "ventas_2025": "Ventas ene-sep 2025 (€)", "variacion_pct": "Variación (%)",
})
salida["Variación (%)"] = salida["Variación (%)"] / 100

with pd.ExcelWriter(ruta, engine="openpyxl") as xls:
    salida.to_excel(xls, sheet_name="Ranking", index_label="Puesto")
    hoja = xls.sheets["Ranking"]
    for celda in hoja[1]:
        celda.font = Font(bold=True)
    for fila in hoja.iter_rows(min_row=2, min_col=3, max_col=4):
        for celda in fila:
            celda.number_format = "#,##0.00 €"
    for (celda,) in hoja.iter_rows(min_row=2, min_col=5, max_col=5):
        celda.number_format = "0.0%"
    for col, ancho in zip("ABCDE", [8, 26, 24, 24, 14]):
        hoja.column_dimensions[col].width = ancho

comprobacion = pd.read_excel(ruta)
print(comprobacion.dtypes)
round(comprobacion["Ventas ene-sep 2026 (€)"].sum(), 2) == round(ranking["ventas_2026"].sum(), 2)
```

RESULTADO

```
Puesto                      int64
Comercial                     str
Ventas ene-sep 2026 (€)     float64
Ventas ene-sep 2025 (€)     float64
Variación (%)               float64
dtype: object
np.True_
```

DESMONTANDO EL COMANDO

pd.ExcelWriter(...) y number_format

`pd.ExcelWriter(ruta, engine="openpyxl")`	Abre un Excel para escribir y deja acceder a la hoja después de `to_excel`, para darle formato. El `with` lo guarda y cierra al terminar.
`index_label="Puesto"`	El índice (1, 2, 3…) se escribe como primera columna, con ese título.
`hoja[1]`	La fila 1, la cabecera: en negrita.
`number_format = "#,##0.00 €"`	El formato de Excel: miles, dos decimales y el símbolo. En el Excel de Raquel, configurado en español, se verá `1.234,56 €`. El valor de la celda sigue siendo un número.
`"0.0%"`	Porcentaje con un decimal. Por eso antes se ha dividido la variación entre 100: Excel guarda 0,123 y muestra 12,3 %.
`pd.read_excel(ruta)`	La segunda vía: leer el Excel recién escrito y comprobar que las columnas son números y que la suma es la misma.

REVISIÓN DE ELENA

Bien resuelto, y has entendido lo importante: el formato es para los ojos, y al Excel van números. El `es()` para el notebook, y el `number_format` para el Excel. Así Raquel puede ordenar, sumar o hacer una tabla dinámica sin pedirte nada.

Dos filas te las va a preguntar. Héctor sale sin ventas en 2026 porque se fue el 30 de junio de 2025, y Celia crece un 173,6 % porque heredó su cartera al entrar. Es el SCD del Volumen 04: las ventas se atribuyen al comercial que tenía asignado el cliente en la fecha de cada venta, y eso es lo justo para un ranking de rendimiento. Pero ese 173,6 % no es mérito de Celia: ponle una nota en el Excel, o compara a Celia con lo que vendía Héctor a esos mismos clientes. Si Raquel quisiera la cartera actual con su historia completa, sería otra consulta (con la versión vigente del cliente); pregúntale cuál quiere. Y el criterio de «Cantabria» es la provincia del cliente, no la zona del comercial: dilo en el correo.

APÉNDICES

Para tener a mano

La chuleta de pandas y lo que viene en el Volumen 07, que abre el Bloque 1: Power BI.

A. Chuleta de pandas

B. Qué viene en el Volumen 07

A. Chuleta de pandas

Leer y mirar

NECESITAS	ESCRIBE
CSV de Marejada	`pd.read_csv(ruta, sep=";", decimal=",", parse_dates=[...], date_format={...})`
Latin-1	`encoding="latin-1"`
Excel con títulos	`pd.read_excel(ruta, skiprows=3)`
JSON anidado	`pd.json_normalize(lista, record_path="items", meta=[...])`
Parquet	`pd.read_parquet(ruta, columns=[...])` · `df.to_parquet(ruta, index=False)`
Desde DuckDB	`con.sql("...").df()`
Mirar	`df.head()` · `df.shape` · `df.dtypes` · `df.describe()` · `df.isna().sum()`
Valores de una columna	`df["col"].value_counts()` · `.nunique()`

Filtrar, transformar, agregar, unir

NECESITAS	ESCRIBE (Y EN SQL)
Columnas	`df[["a", "b"]]` (`SELECT a, b`)
Filtrar	`df[(df["a"] == 1) & (df["b"] > 2)]` (`WHERE`) · `.isin([...])` (`IN`) · `.query("...")`
Filtrar y elegir columnas	`df.loc[máscara, ["a", "b"]]`
Nulos	`.isna()` · `.notna()` · `.fillna(x)` (`coalesce`)
Columna nueva	`df["c"] = df["a"] * df["b"]`
Texto y fechas	`.str.strip().str.lower()` · `.dt.year` · `.dt.to_period("M")`
Tipo	`.astype(int)` · `.astype("Int64")` · `pd.to_numeric(x, errors="coerce")` (`TRY_CAST`)
Traducir valores	`.map(diccionario)`
Agrupar	`df.groupby("a").agg(n=("b", "count"), total=("c", "sum"))` (`GROUP BY`)
Tabla dinámica	`pd.pivot_table(df, index=..., columns=..., values=..., aggfunc=...)`
Ordenar	`.sort_values("a", ascending=False)` (`ORDER BY ... DESC`)
Unir	`a.merge(b, on="k", how="left", validate="many_to_one")` (`LEFT JOIN`)
Anti-join	`merge(..., how="left", indicator=True)` y `_merge == "left_only"`
Apilar	`pd.concat([a, b])` (`UNION ALL`)
Por periodo	`s.resample("MS").sum()` con fechas en el índice
Ventanas	`.rolling(3).mean()` · `.shift(1)` (`lag`) · `.cumsum()` · `.pct_change()`
SQL sobre un DataFrame	`duckdb.sql("SELECT ... FROM df").df()`

APIs, gráficos y salida

NECESITAS	ESCRIBE
Petición	`r = requests.get(url, params={...}, timeout=10)`
Comprobar	`r.raise_for_status()` · `r.status_code` · `r.json()`
Errores de red	`except requests.exceptions.RequestException:`
Cortesía	`time.sleep(1)` entre peticiones · caché en un fichero
Gráfico	`fig, ax = plt.subplots()` · `ax.plot(x, y)` · `ax.set_title(...)`
Guardar	`fig.savefig(ruta, dpi=150, bbox_inches="tight")` antes de `plt.show()`
Formato español	formatear con `:,.2f` y cambiar `,` y `.` (MJD-606)
Excel	`df.to_excel(ruta, index=False)` · `pd.ExcelWriter` para dar formato

Las trampas de este volumen

TRAMPA	CÓMO SE EVITA
`and` / `or` o sin paréntesis en una máscara	`(cond1) & (cond2)`, cada condición entre paréntesis
Enteros con nulos que salen como `1.0`	`.astype("Int64")`
`merge` que multiplica filas	`validate="many_to_one"` (o `one_to_one`)
Analizar desde bronze	Partir siempre de silver o de gold
`DECIMAL` que llega como `float64`	Las sumas al céntimo, en DuckDB
Código antiguo con `.copy()` y avisos	En pandas 3.0 un filtro siempre es una copia
`.dt.dayofweek` con el lunes en 0	Recordarlo, o usar `isodow` en SQL
Una API que responde con error y no lo miras	`raise_for_status()` siempre
Peticiones sin `timeout`	Ponerlo siempre
La API devuelve fechas que no pediste	Filtrar lo que vuelve, no fiarse de lo que se pidió
Claves de una API en el código	Variables de entorno, nunca en GitHub
`savefig` después de `show`	Guardar antes
Números con formato español en un Excel	Al Excel van números; el formato, con `number_format`

B. Qué viene en el Volumen 07

Con este volumen termina el **Bloque 0**: SQL, modelado y Python. El Volumen 07 abre el **Bloque 1, Power BI**, y el camino al primer examen, el PL-300:

- **Instalar Power BI Desktop en tu Mac**, que es lo más delicado del bloque: solo existe para Windows, así que irá en una máquina virtual de Windows 11 para ARM con VMware Fusion Pro (gratuito), con 4 GB de tus 8 para la máquina virtual. Y el plan B, Power BI en el navegador.
- **Power Query**: la herramienta de Power BI para leer y limpiar datos. Vas a reconocer en ella casi todo lo de los Volúmenes 02 y 06 (tipos, filtros, combinar, agrupar, pivotar), ahora con ventanas y botones, y un lenguaje por debajo, M.
- Cargar **gold** en Power BI: la dimensión de fechas, las relaciones y el esquema en estrella del Volumen 04, tal como los pide el examen.

UN POSIBLE COSTE, ANTES DE EMPEZAR

Para la máquina virtual de Windows puede hacer falta una **licencia de Windows 11**, que es de pago. El Volumen 07 empieza explicando las opciones y lo que cuesta cada una, para que decidas **antes** de instalar nada.

ANTES DE PASAR AL VOLUMEN 07

Tienes que poder hacer esto sin mirar: leer un CSV de Marejada con `read_csv` y sus opciones, filtrar con dos condiciones, agrupar con `agg` y nombres, un `merge` con `validate` y un anti-join con `indicator`, y pasar un DataFrame a DuckDB y de vuelta. Y los seis tickets en `notebooks/`, con los entregables en `entregables/`, en GitHub. Revisa también la cuenta de estudiante para la prueba de Fabric: el Bloque 2 la necesita.